



Universidad de Castilla-La Mancha

Escuela Superior de Ingeniería Informática de Albacete

Escuela Superior de Informática de Ciudad Real

Trabajo Fin de Máster

Máster Universitario en Inteligencia Artificial

Uso de estrategias y algoritmos de IA para el control de drones autónomos condicionados a un plan de vuelo

Teresa Valero Díaz

Julio, 2026

TRABAJO FIN DE MÁSTER

Máster Universitario en Inteligencia Artificial

Uso de estrategias y algoritmos de IA para el control de drones autónomos condicionados a un plan de vuelo

Autor: Teresa Valero Díaz

Director: María Julia Flores Gallego

Codirector: Rafael Casado González

*Dedico este Trabajo Fin de Máster a mi familia,
que, como los drones, me apoyaron en mis caídas
y dificultades, hasta poder alzar el vuelo*

Declaración de autoría

Yo, Teresa Valero Díaz, con DNI 49213759S, declaro que soy la única autora del trabajo fin de máster titulado “Uso de estrategias y algoritmos de IA para el control de drones autónomos condicionados a un plan de vuelo”, que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores.

Albacete, a 7 de julio de 2026

Fdo.: Teresa Valero Díaz

Resumen

El auge de la movilidad aérea avanzada y el desarrollo de aeronaves no tripuladas de gran envergadura, como los aerotaxis, exigen sistemas de control autónomo altamente sofisticados, capaces de operar frente a dinámicas no lineales complejas y entornos estocásticos. En respuesta a este reto, el presente trabajo propone el diseño y desarrollo de un marco de control basado en técnicas de Aprendizaje por Refuerzo Profundo.

Para ello, se implementa el algoritmo *Proximal Policy Optimization* (PPO) mediante una arquitectura Actor-Crítico asimétrica dentro del simulador Isaac Lab. El proyecto aborda la resolución secuencial de dos problemas cinemáticos de alta dimensionalidad: en primer lugar, la estabilización estática de la aeronave (*hovering*) y, posteriormente, el seguimiento dinámico de trayectorias espaciales variables (*tracking*). Con el objetivo de garantizar la robustez del sistema y garantizar la obtención de políticas que permitan un despliegue físico real (mitigando la brecha *Sim-to-Real*), el entorno de entrenamiento incorpora modificaciones en su núcleo físico y se han implementado diversas técnicas de aleatorización de dominio, exponiendo al vehículo a variaciones continuas de masa aerodinámica y ráfagas de viento.

Asimismo, dada la extrema sensibilidad de estos algoritmos a su parametrización inicial, este trabajo aporta como valor añadido la creación de un *pipeline* computacional íntegramente automatizado. Este flujo de trabajo orquesta la generación, el entrenamiento paralelo y la evaluación estadística de los modelos mediante técnicas de *Random Search*. En conjunto, los resultados obtenidos demuestran la viabilidad técnica de emplear este enfoque para dominar la navegación de cuadricópteros de estas dimensiones, estableciendo una metodología rigurosa y escalable que carece de equivalentes documentados en el estado del arte actual.

Agradecimientos

En primer lugar, me gustaría agradecer a mis directores de este proyecto, María Julia Flores Gallego, por haberme enseñado todo cuanto sé, y a Rafael Casado González, por haberme animado a realizar este proyecto.

También, quiero agradecer especialmente a Víctor Blázquez Milara por introducirme en el mundo de los simuladores Isaac, por facilitarme todo el material y código fuente que se tomó como base y que fue imprescindible para comenzar la elaboración de este proyecto, y especialmente, por haberme enseñado todo cuanto sabía con una paciencia infinita.

Por último, quiero agradecer a mi novio, a mi familia y amigos, que desde el primer día se interesaron por este proyecto y me apoyaron y animaron a seguir adelante, incluso en los momentos más difíciles.

Índice general

1	Introducción	1
1.1	Motivación	2
1.2	Objetivos y competencias	3
1.3	Metodología	3
1.4	Estructura del trabajo	4
2	Estado del arte	7
2.1	Control de vehículos aéreos no tripulados (UAV)	7
2.1.1	Fundamentos físicos	7
2.1.2	Tipos de problemas en el control de UAV	10
2.1.3	Definición de plan de vuelo	10
2.2	Aprendizaje por refuerzo	11
2.2.1	Conceptos básicos	12
2.2.2	Tipos de algoritmos	12
2.2.3	Aprendizaje por refuerzo profundo	13
2.2.4	Proximal Policy Optimization	13
2.3	Aprendizaje por refuerzo aplicado a UAV	14
2.3.1	Navegación autónoma	14
2.3.2	Control de actitud	15
2.3.3	Entrenamiento y simulación	15
2.3.4	Evaluación de controladores	16
2.4	Simulador Isaac de NVIDIA	16
2.4.1	Isaac Sim	18
2.4.2	Isaac Lab	18

3	Planteamiento y definición de los problemas de control	19
3.1	Modelo de dron empleado	19
3.2	Objetivos del dron	20
3.3	Definición del problema de estabilidad	21
3.3.1	<i>Configuración del entorno</i>	21
3.3.2	<i>Espacio de observaciones y acciones</i>	22
3.3.3	<i>Configuración de PPO</i>	23
3.3.4	<i>Estructura de recompensas y terminaciones</i>	27
3.3.5	<i>Técnicas de prevención de sobreajuste</i>	29
3.4	Definición del problema de seguimiento de rutas	31
3.4.1	<i>Configuración del entorno</i>	31
3.4.2	<i>Espacio de observaciones y acciones</i>	32
3.4.3	<i>Configuración de PPO</i>	33
3.4.4	<i>Estructura de recompensas y terminaciones</i>	34
3.4.5	<i>Técnicas de prevención del sobreajuste</i>	35
4	Experimentación y resultados	37
4.1	Experimentos definidos para la estabilidad	37
4.2	Resultados de la experimentación para la estabilidad	41
4.3	Experimentos definidos para seguimiento del plan de vuelo	45
4.4	Resultados de la experimentación para el seguimiento del plan de vuelo	47
5	Conclusiones	51
A	Anexos	53
A.1	Código del proyecto	53
A.2	Otros experimentos realizados	53
A.3	Comportamientos detectados en la fase de entrenamiento	59
	Referencia bibliográfica	64

Índice de figuras

3.1	Modelo de dron empleado.	19
3.2	Visualización de un dron sobre su punto rojo de referencia.	22
3.3	Visualización de un dron siguiendo su punto rojo de referencia con los marcadores de dirección del dron y el ideal.	32
4.1	Diagrama global del flujo de trabajo automatizado diseñado para la experimentación en Isaac Lab.	38
4.2	Arquitectura del flujo de trabajo para la generación paramétrica y ejecución secuencial de los entrenamientos.	38
4.3	Flujo de procesamiento de datos para la evaluación masiva, extracción de métricas escaladas y selección estadística del modelo óptimo.	39
4.4	Distribución de la recompensa media obtenida por cada uno de los 35 modelos durante la fase de entrenamiento.	42
4.5	Curvas de evolución temporal de la recompensa durante el entrenamiento, filtradas exclusivamente para los cinco mejores modelos resultantes de la fase de validación (Test).	42
4.6	Análisis del sobreajuste (<i>overfitting</i>): comparativa entre la recompensa bruta de entrenamiento (barras sólidas) y el rendimiento en los entornos de evaluación paralelos (barras rayadas).	43
4.7	Análisis mediante gráficos de radar del rendimiento multidimensional. Se expone la invariabilidad de los mejores modelos frente al incremento masivo de instanciación paralela (256, 1024 y 8196 entornos de prueba).	45
4.8	Distribución de la recompensa media obtenida por cada uno de los 35 modelos durante la fase de entrenamiento de <i>tracking</i>	47
4.9	Curvas de evolución temporal de la recompensa durante el entrenamiento, filtradas para los cinco mejores modelos resultantes de la evaluación de <i>Tracking</i>	48
4.10	Análisis del sobreajuste en seguimiento dinámico: comparativa entre la recompensa de entrenamiento y el rendimiento en validación.	48

4.11	Análisis mediante gráficos de radar para evaluar la eficacia predictiva, el desempeño dinámico y la consistencia operativa de los modelos de <i>Tracking</i> al escalar el estrés computacional (256, 1024 y 8196 entornos).	50
4.12	Seguimiento de una ruta aleatoria.	50
A.1	Parte I. Selección de experimentación realizada para hover.	54
A.2	Parte II. Selección de experimentación realizada para hover.	55
A.3	Parte III. Selección de experimentación realizada para hover.	56
A.4	Parte IV. Selección de experimentación realizada para hover.	57
A.5	Selección de experimentación realizada para tracking.	58
A.6	Comportamientos identificados durante la fase de entrenamiento en hover y tracking.	59

Índice de tablas

1.1	Planificación de sprints y resultados del proyecto.	5
2.1	Dinámica de los movimientos y fuerzas de sustentación del UAV.	8
2.2	Reglas físicas del movimiento (Ecuaciones de Newton-Euler).	9
2.3	Dimensiones y métricas utilizadas para la evaluación del agente PPO. . .	17
3.1	Definición del espacio de acciones y restricciones físicas del entorno. . .	24
3.2	Definición del espacio de observaciones, unidades y ruido aplicado. . . .	24
3.3	Espacio de búsqueda de hiperparámetros de PPO para el algoritmo Random Search.	25
3.4	Espacio de observaciones, unidades y ruido aplicado para el problema de seguimiento de rutas.	33
3.5	Comparación entre recompensas de hovering y seguimiento de trayectoria. .	35
4.1	Cálculo de las métricas en Hovering.	40
4.2	Resultados globales de la evaluación multidimensional para los 5 mejores modelos de vuelo estacionario, ordenados por su <i>Score Global</i> en Test. . .	44
4.3	Configuración de hiperparámetros para los 5 modelos más destacados en la tarea de vuelo estacionario (<i>Hovering</i>).	44
4.4	Cálculo de las métricas en Tracking.	46
4.5	Resultados globales de la evaluación multidimensional para los 5 mejores modelos de seguimiento (<i>Tracking</i>), ordenados por su <i>Score Global</i> en Test. .	49
4.6	Configuración de hiperparámetros para los cinco modelos más destacados en la tarea de seguimiento de trayectorias dinámicas (<i>Tracking</i>).	49

1. Introducción

El control autónomo de drones constituye una de las áreas más activas dentro de la Inteligencia Artificial aplicada al control de sistemas dinámicos, debido a su amplio rango de aplicaciones en ámbitos como la logística, la inspección industrial, la vigilancia o la exploración de entornos complejos. En este contexto, las técnicas de aprendizaje por refuerzo profundo han emergido como una alternativa prometedora frente a los enfoques clásicos de control, al permitir el aprendizaje de políticas complejas directamente a partir de la interacción con el entorno [Sutton et al., 1998].

En este trabajo se ha diseñado e implementado un controlador predictivo basado en Inteligencia Artificial capaz de generar las velocidades angulares de los rotores de un dron a partir del estado de la aeronave, un plan de vuelo completo y el tiempo actual. A diferencia de los enfoques puramente reactivos, este modelo incorpora información temporal y estructural de una misión preestablecida previamente por un sistema externo y autónomo, permitiendo que la aeronave se anticipe a maniobras futuras [Camacho and Bordons, 2013] y se adapte dinámicamente a la evolución del entorno. Así, este sistema se enfoca por tanto en la autonomía de ejecución, gestionando por sí mismo las velocidades angulares de los rotores para compensar desviaciones sin intervención humana.

El entrenamiento del controlador se ha llevado a cabo en entornos de simulación física avanzada, concretamente mediante el uso de NVIDIA Isaac Sim e Isaac Lab, lo que permite disponer de un entorno altamente realista, reproducible y configurable. En particular, se han introducido técnicas de *domain randomization* sobre variables clave como el viento, la masa del dron, su posición y orientación inicial, así como ruido en los sensores, con el objetivo de mejorar la capacidad de generalización del modelo [Tobin et al., 2017, NVIDIA, 2026].

Por otro lado, se ha desarrollado un pipeline automático de experimentación que integra un algoritmo de *random search* [Bergstra and Bengio, 2012] para la optimización de hiperparámetros del algoritmo *Proximal Policy Optimization* (PPO) [Schulman et al., 2017]. Este

pipeline permite definir configuraciones experimentales de forma flexible desde una interfaz simplificada, automatizando completamente la ejecución, entrenamiento y evaluación de múltiples experimentos sin necesidad de modificar el código base. Esta contribución facilita la exploración sistemática del espacio de configuraciones y mejora la reproducibilidad del proceso experimental.

1.1. Motivación

El control de vehículos aéreos no tripulados presenta desafíos significativos debido a la naturaleza altamente no lineal de su dinámica, la presencia de perturbaciones externas y la necesidad de operar en entornos cambiantes e inciertos [Khalil and Grizzle, 2002]. Los métodos tradicionales de control, como los controladores PID o los enfoques basados en modelos, requieren una modelización precisa del sistema y suelen presentar limitaciones cuando se enfrentan a escenarios complejos o no previstos.

En este contexto, el aprendizaje por refuerzo profundo ofrece una alternativa atractiva, ya que permite a los agentes aprender estrategias de control directamente a partir de la experiencia [Sutton et al., 1998], sin necesidad de disponer de un modelo explícito del sistema. Sin embargo, estos métodos presentan también retos importantes, como la elevada sensibilidad a los hiperparámetros [Henderson et al., 2018], la dificultad de generalización y el alto coste computacional asociado al entrenamiento.

La motivación principal de este trabajo surge de la necesidad de abordar estas limitaciones mediante el diseño de un sistema robusto, capaz de generalizar a distintas condiciones de operación y que facilite la experimentación sistemática. En particular, se busca mejorar la eficiencia del proceso de entrenamiento y optimización mediante la automatización de experimentos y la incorporación de técnicas como el *domain randomization* [Tobin et al., 2017] y la búsqueda aleatoria de hiperparámetros.

Asimismo, se pretende avanzar en el desarrollo de controladores predictivos que no solo respondan al estado actual del sistema, sino que sean capaces de anticipar la evolución futura de la misión a partir del plan de vuelo, lo que resulta especialmente relevante en tareas de seguimiento de trayectorias complejas. Además, el uso de modelos de Inteligencia Artificial para el control de bajo nivel no es solo una alternativa al control lineal clásico, sino una necesidad de futuro al ofrecer una flexibilidad superior para gestionar drones con dinámicas complejas o configuraciones no convencionales, donde los modelos matemáticos tradicionales se vuelven difíciles de ajustar.

1.2. Objetivos y competencias

El objetivo principal de este trabajo es diseñar, implementar y evaluar un controlador autónomo para drones basado en técnicas de aprendizaje por refuerzo profundo, capaz de seguir trayectorias definidas mediante planes de vuelo en entornos simulados complejos. Así, se definen los siguientes objetivos específicos:

- **OBJ_1:** Diseñar un modelo empleando técnicas y estrategias de IA que reciba como entrada el estado de la aeronave, el plan de vuelo y el tiempo actual, y genere las velocidades angulares de los rotores.
- **OBJ_2:** Implementar un entorno de simulación que permita entrenar y validar el controlador en condiciones realistas y reproducibles.
- **OBJ_3:** Analizar la capacidad predictiva, la estabilidad y la eficiencia del modelo en el seguimiento de trayectorias aéreas complejas.
- **OBJ_4:** Evaluar la robustez del sistema ante perturbaciones externas o variaciones del plan de vuelo.

Este trabajo contribuye al desarrollo de diversas competencias del Máster Universitario en Inteligencia Artificial:

- **CP01:** Aplicar conceptos, técnicas y métodos avanzados de Inteligencia Artificial en entornos que requieran aprendizaje automático, razonamiento y percepción.
- **CP02:** Planificar y ejecutar proyectos de desarrollo y despliegue de soluciones a problemas complejos que requieran de técnicas y métodos de Inteligencia Artificial en entornos que impliquen conducta inteligente, aprendizaje e interacción.
- **CP03:** Aplicar técnicas de análisis, estadística e Inteligencia Artificial sobre datos, para dar apoyo a la toma de decisiones, descubriendo relaciones, y ofreciendo una visualización y comprensión intuitiva, precisa y profunda.

1.3. Metodología

Para garantizar la agilidad y la finalización del proyecto en el tiempo disponible, se ha empleado la metodología ágil de SCRUM. Dado que el entrenamiento de modelos empleando Aprendizaje por Refuerzo es un proceso inherentemente iterativo y experimental, el marco de trabajo ágil permite reajustar las funciones de recompensa y los hiperparámetros de manera sistemática tras cada sprint de entrenamiento, facilitando una evolución controlada del

comportamiento del agente hasta alcanzar la estabilidad deseada. Así, se ha dividido el trabajo en sprints de 1 semana de duración, estructurando el marco de trabajo de la siguiente forma:

- **Reunión de planificación:** Cada mes, se ha realizado una reunión para acordar las tareas que se llevarán a cabo en los sprints semanales de ese mes.
- **Sprint:** Con duración de 1 semana, se realizan las tareas especificadas en la reunión de planificación, elaborando un informe de tareas realizadas en cada sprint.
- **Revisión del sprint y retrospectiva:** El mismo día que se realiza la reunión de planificación, se ha realizado una reunión previa de revisión del sprint y retrospectiva de las tareas realizadas.

Así, se ha organizado el proyecto en sprints semanales con una dedicación aproximada de 15 horas por sprint, hasta un total de 150 horas. El proyecto se desarrolló a lo largo de 10 semanas, desde el 23 de diciembre hasta el 1 de marzo. En la Tabla 1.1 se detalla la planificación de los sprints.

1.4. Estructura del trabajo

El presente documento se organiza en los siguientes capítulos:

- **Capítulo 1:** Se introduce una visión general del proyecto desarrollado, los objetivos y competencias planteados, la metodología que se ha seguido para su correcto desarrollo y se explica el contenido de los capítulos contenidos en esta memoria.
- **Capítulo 2:** Se presenta el estado del arte, donde se revisan los fundamentos del control de UAV, los conceptos clave del aprendizaje por refuerzo y su aplicación al control de drones, así como las herramientas de simulación empleadas.
- **Capítulo 3:** Se describe el planteamiento del problema, incluyendo la definición de los modelos de dron utilizados, los objetivos de control y la formulación de los problemas de estabilidad y seguimiento de trayectorias. También se detallan las configuraciones del entorno, las observaciones, acciones y funciones de recompensa. En este capítulo se desarrollan los objetivos **OBJ_1** y **OBJ_2**.
- **Capítulo 4:** Se expone la experimentación realizada y los resultados obtenidos, tanto en el problema de estabilidad como en el seguimiento de planes de vuelo, incluyendo el análisis del impacto de los hiperparámetros y del uso de técnicas de aleatorización. Así, en este capítulo se aborda la implementación de los objetivos **OBJ_3** y **OBJ_4**.

- **Capítulo 5:** En este último capítulo se presentan las conclusiones del trabajo, así como posibles líneas futuras de investigación.

Finalmente, se incluyen anexos con información adicional relevante y la bibliografía utilizada.

Tabla 1.1: Planificación de sprints y resultados del proyecto.

Sprint	Fechas	Objetivo principal	Resultado obtenido
1	23/12 – 29/12	Comprensión del problema y definición inicial de tareas	Definición clara del problema y alcance del sistema
2	30/12 – 05/01	Estudio de Isaac Sim e Isaac Lab	Entorno de simulación comprendido y primeros ejemplos ejecutados
3	06/01 – 12/01	Diseño del sistema y arquitectura PPO	Arquitectura definida y diseño del pipeline planteado
4	13/01 – 19/01	Implementación de Random Search	Algoritmo de búsqueda de hiperparámetros funcional
5	20/01 – 26/01	Desarrollo del pipeline automático	Sistema automatizado de experimentación implementado
6	27/01 – 02/02	Desarrollo del entorno de estabilidad (hover)	Entorno de entrenamiento básico con recompensas definidas
7	03/02 – 09/02	Refinamiento del entorno de estabilidad	Modelo estable funcional tras múltiples experimentos
8	10/02 – 16/02	Implementación del seguimiento de puntos fijos	Primer entorno de seguimiento y validación inicial
9	17/02 – 23/02	Implementación del seguimiento de trayectorias	Entorno con trayectorias dinámicas y experimentación comparativa
10	24/02 – 01/03	Evaluación final y análisis de resultados	Resultados analizados y validación del sistema completa

2. Estado del arte

2.1. Control de vehículos aéreos no tripulados (UAV)

El control de vehículos aéreos no tripulados es uno de los retos más relevantes en el campo de la robótica aérea, dado que de éste depende la estabilidad del sistema y su capacidad para ejecutar trayectorias de forma autónoma [Cai et al., 2011]. En particular, los drones tipo multirrotor presentan una dinámica no lineal (esto significa que sus movimientos no cambian de forma simple o proporcional) y fuertemente acoplada (lo que implica que un movimiento, como inclinarse, afecta inevitablemente a otros aspectos, como la altura) [Mahony et al., 2012]. Esto supone un desafío significativo para el diseño de algoritmos de control robustos, y, para ello, es necesario comprender cuáles son sus fundamentos físicos y qué tipos de problemas de control hay.

2.1.1. Fundamentos físicos

El vuelo de un cuadricóptero se rige por la dinámica de cuerpos rígidos, una rama de la física que estudia el movimiento de objetos que no se deforman. El dron opera con seis grados de libertad (6-DoF), de los cuales tres son movimientos de traslación (moverse hacia adelante, hacia los lados o hacia arriba) y los otros tres son movimientos de rotación (girar sobre sí mismo).

Sin embargo, es un sistema subactuado, ya que solo tiene cuatro rotores para controlar esos seis movimientos. Esto significa que el dron no puede moverse en cualquier dirección de forma independiente. Por ejemplo, para poder avanzar está obligado a inclinarse primero [Beard and McLain, 2012]. Las relaciones cinemáticas y los principios de operación derivados de esta configuración se detallan en la Tabla 2.1.

Tabla 2.1: Dinámica de los movimientos y fuerzas de sustentación del UAV.

Concepto	Acción de los motores y fórmula	Explicación del fenómeno
Fuerza por rotor (F_i)	$F_i = k_f \omega_i^2$	Cada rotor genera una fuerza individual proporcional al cuadrado de su velocidad de giro (ω). k_f es la constante de empuje del rotor [Luukkonen, 2011].
Sustentación total (T)	$T = k_f \sum_{i=1}^4 \omega_i^2$	Es la suma de las fuerzas de los cuatro rotores. Es la fuerza principal que empuja al dron hacia arriba.
Condición de vuelo estacionario	$T = mg$	Para elevarse, el empuje total (T) debe superar el peso (masa m por gravedad g). En equilibrio, el dron flota a una altura constante [Beard and McLain, 2012].
Alabeo (Roll) y Cabeceo (Pitch)	$\begin{aligned} \tau_\phi &= lk_f(\omega_2^2 - \omega_4^2) \\ \tau_\theta &= lk_f(\omega_3^2 - \omega_1^2) \end{aligned}$	Al variar la velocidad de rotores opuestos, el dron se inclina. Esta inclinación redirige parte del empuje hacia los lados para generar movimiento.
Guiñada (Yaw - τ_ψ)	$\tau_\psi = k_m(\omega_1^2 - \omega_2^2 + \omega_3^2 - \omega_4^2)$	Se basa en el par de torsión o reacción aerodinámica que hace que el dron gire sobre su propio eje vertical [Nonami et al., 2010].

Nomenclatura de variables:

k_f, k_m : Coeficientes aerodinámicos de empuje y de par, respectivamente.

ω_i : Velocidad angular del motor i .

m : Masa total de la aeronave.

l : Distancia desde el centro del UAV al eje del motor (longitud del brazo).

g : Aceleración de la gravedad (9.81 m/s^2).

Para la obtención de estas ecuaciones, se asume el empleo de un cuadricóptero ideal con cuatro hélices exactamente iguales y distribuidas de forma homogénea con respecto al centro de masas. De esta forma, se asumirá que la longitud del brazo (l) es constante para todos los motores y que los coeficientes aerodinámicos (k_f, k_m) son idénticos.

Por otro lado, para que el sistema de control sepa exactamente qué instrucciones enviar a los rotores, se utilizan las ecuaciones de Newton-Euler [Mahony et al., 2012], que dividen el movimiento en dos partes fundamentales: el desplazamiento, que describe cómo se mueve el dron de un punto a otro en el espacio, y el giro, que describe cómo el dron cambia sus ángulos de inclinación. El modelo matemático integral que representa ambos comportamientos se presenta en la Tabla 2.2.

Tabla 2.2: Reglas físicas del movimiento (Ecuaciones de Newton-Euler).

Concepto	Representación Matemática	Explicación Simplificada
Desplazamiento (Traslación)	$m\ddot{r} = R \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix}$	El movimiento en el espacio depende de la orientación del dron (matriz R) y de si la fuerza de empuje (T) logra vencer al peso (mg).
Giro (Rotación)	$I\dot{\Omega} + \Omega \times (I\Omega) = \tau$	La rapidez con la que cambia el ángulo de inclinación depende de la inercia (I) y del torque o fuerza de giro aplicada (τ) por los rotores.

Nomenclatura de variables:

m : Masa total del UAV.

$\ddot{r}$: Aceleración lineal del vehículo.

R : Matriz de rotación (orientación del dron con respecto a la tierra).

T : Empuje total generado por los rotores.

mg : Fuerza del peso (masa por gravedad).

I : Tensor de inercia (resistencia al giro).

$\Omega, \dot{\Omega}$: Velocidad y aceleración angular, respectivamente.

τ : Vector de torques o momentos de fuerza aplicados.

2.1.2. Tipos de problemas en el control de UAV

En el ámbito de la robótica aérea, los objetivos de control se clasifican principalmente en dos categorías según la naturaleza de la referencia que el vehículo debe seguir. Estos problemas determinan la complejidad del algoritmo de control y cómo este debe gestionar las fuerzas y momentos generados por los rotores. Así, encontramos dos tipos de problemas:

1. **Problema de estabilidad o *Hover*.** El control de estabilidad tiene como objetivo mantener al UAV en una posición (x, y, z) y una orientación (ψ) constantes respecto a un punto de referencia fijo en el aire. Por lo tanto, el error entre la posición actual y el punto de referencia debe tender a cero, contrarrestando perturbaciones externas como ráfagas de viento o el ruido inherente de los sensores [Mahony et al., 2012]. Para lograrlo, el controlador debe garantizar que el empuje total de los rotores sea exactamente igual al peso de la aeronave, manteniendo una inclinación nivelada, de forma que la sustentación total del dron cumpla con la condición de vuelo estacionario, descrita en la Tabla 2.1.
2. **Seguimiento de trayectorias o *Tracking*.** A diferencia del vuelo estacionario, el seguimiento de trayectorias es un problema dinámico donde el punto de referencia que debe seguir el dron varía en función del tiempo, de forma que para cada instante t el dron debe encontrarse en una posición específica. En este escenario, el controlador no solo debe minimizar el error de posición, sino que también debe anticipar y gestionar las velocidades y aceleraciones necesarias para alcanzar la posición correcta dentro de la ruta en el instante preciso [Beard and McLain, 2012]. Este problema es considerablemente más complejo debido al acoplamiento dinámico, puesto que para ganar velocidad y seguir la trayectoria, el dron debe inclinarse, lo que reduce la fuerza de sustentación total y obliga al sistema de control a compensar esa pérdida para no perder altura mientras se desplaza [Cai et al., 2011].

2.1.3. Definición de plan de vuelo

Un plan de vuelo en un vehículo aéreo no tripulado puede entenderse como la descripción formal de la trayectoria que el vehículo debe seguir para completar una misión. Esta trayectoria puede representarse de diferentes formas según el nivel de detalle y las necesidades del sistema de control. La aproximación más común consiste en definir una secuencia discreta de puntos de paso (*waypoints*), donde cada punto representa una posición objetivo en el espacio [Farid et al., 2018]. Sin embargo, en aplicaciones de control autónomo y aprendizaje por refuerzo es habitual emplear representaciones continuas de trayectoria, ya que

permiten generar referencias más suaves y mejor adaptadas a las limitaciones dinámicas del vehículo [Park et al., 2021].

Dentro de este enfoque, una trayectoria puede modelarse como una función continua del tiempo, $p(t)$, que define la posición de una referencia móvil a lo largo de la misión. A diferencia de los enfoques basados en puntos fijos, esta representación proporciona una referencia dinámica que evoluciona de forma continua, permitiendo al controlador trabajar con objetivos cambiantes sin introducir discontinuidades en las señales de control. Este tipo de planteamiento resulta especialmente útil en escenarios donde se busca evaluar la capacidad de adaptación del UAV frente a variaciones constantes en su objetivo de seguimiento [Kober et al., 2013].

En algunos entornos de entrenamiento y validación, estas trayectorias continuas pueden generarse de forma dinámica dentro de regiones espaciales acotadas, permitiendo definir patrones de movimiento variables y no deterministas. Una estrategia habitual consiste en restringir la evolución de la trayectoria dentro de un volumen de vuelo predefinido, de manera que cuando la referencia alcanza un límite espacial, su dirección se reajusta de forma suave para mantener la continuidad del movimiento. Este mecanismo permite generar trayectorias dinámicas que conservan propiedades de suavidad y continuidad [Shi et al., 2024], evitando cambios bruscos que puedan dificultar el seguimiento por parte del controlador.

Además, la representación continua de trayectorias facilita la incorporación de restricciones espaciales y dinámicas durante la fase de planificación. Mediante la limitación de parámetros como velocidad, aceleración o cambios de dirección, es posible garantizar que las referencias generadas sean compatibles con las capacidades físicas del UAV. En este sentido, la planificación continua no solo mejora la calidad del seguimiento, sino que también permite construir entornos de entrenamiento más realistas y exigentes, donde el sistema de control debe mantener estabilidad y precisión frente a objetivos dinámicos, en línea con los principios de planificación multi-restringida [Casado et al., 2026].

2.2. Aprendizaje por refuerzo

El aprendizaje por refuerzo es una de las áreas más relevantes dentro del aprendizaje automático para problemas de toma de decisiones secuenciales. A diferencia de otros paradigmas, como el aprendizaje supervisado o no supervisado, en aprendizaje por refuerzo el agente aprende mediante interacción directa con un entorno, ajustando su comportamiento en función de las recompensas obtenidas. Este enfoque ha demostrado ser especialmente útil en robótica y sistemas autónomos, donde las decisiones deben tomarse de forma con-

tinua y adaptativa en entornos dinámicos [Kober et al., 2013].

2.2.1. Conceptos básicos

El aprendizaje por refuerzo modela el problema de aprendizaje como la interacción entre un agente y un entorno. En cada instante de tiempo, el agente observa el estado actual del entorno, selecciona una acción y recibe una recompensa que evalúa la calidad de dicha decisión. A partir de esta información, el agente actualiza su estrategia de actuación con el objetivo de maximizar la recompensa acumulada a largo plazo [Sutton et al., 1998].

Formalmente, este proceso suele representarse mediante un Proceso de Decisión de Markov (MDP), definido como una tupla (S, A, P, R, γ) , donde (S) representa el espacio de estados, (A) el conjunto de acciones disponibles, (P) la dinámica de transición entre estados, (R) la función de recompensa y (γ) el factor de descuento. Este modelo permite formalizar problemas donde las decisiones actuales influyen en estados futuros y en la recompensa total esperada.

Uno de los principales retos en el aprendizaje por refuerzo es el equilibrio entre exploración y explotación. La exploración consiste en probar nuevas acciones para descubrir comportamientos potencialmente mejores, mientras que la explotación utiliza el conocimiento ya adquirido para maximizar la recompensa inmediata. Encontrar un equilibrio adecuado entre ambas estrategias es fundamental para lograr un aprendizaje eficiente [Sutton et al., 1998].

2.2.2. Tipos de algoritmos

Los algoritmos de aprendizaje por refuerzo pueden clasificarse según la forma en que representan y optimizan la política de actuación. Una primera clasificación distingue entre métodos basados en valor y métodos basados en política [Sutton et al., 1998].

Los métodos basados en valor buscan estimar la utilidad esperada de ejecutar una acción en un estado determinado. Algoritmos como Q-Learning o SARSA pertenecen a esta categoría y han sido ampliamente utilizados en problemas discretos debido a su simplicidad y eficacia.

Por otro lado, los métodos basados en política optimizan directamente la política del agente, es decir, la función que determina qué acción ejecutar en cada estado. Este enfoque resulta especialmente útil en espacios de acción continuos, donde la estimación explícita de valores puede resultar poco práctica.

Además, existe una familia híbrida conocida como métodos actor-crítico, que combina ambos enfoques. En estos métodos, un componente actor se encarga de actualizar la po-

lítica, mientras que un componente crítico evalúa la calidad de las acciones tomadas. Este esquema ha demostrado una gran eficiencia en problemas de control continuo y robótica [Konda and Tsitsiklis, 1999, Schulman et al., 2017].

2.2.3. Aprendizaje por refuerzo profundo

Aunque los métodos clásicos de aprendizaje por refuerzo son efectivos en espacios de estado pequeños o discretos, presentan limitaciones cuando el entorno es complejo o de alta dimensionalidad. Para abordar este problema surge el aprendizaje por refuerzo profundo, que combina aprendizaje por refuerzo con redes neuronales profundas como aproximadores de funciones [Vincent et al., 2018].

El uso de redes neuronales permite representar funciones de valor o políticas sobre espacios continuos y de gran dimensión, como imágenes, señales sensoriales o estados físicos complejos. Esto ha ampliado considerablemente el campo de aplicación del aprendizaje por refuerzo, especialmente en robótica y control autónomo.

Sin embargo, el uso de aproximadores profundos introduce nuevos retos, como la estabilidad del entrenamiento, la sensibilidad a hiperparámetros y la necesidad de interacción masiva con el entorno. Por este motivo, en aplicaciones robóticas suele recurrirse a simulación intensiva para entrenar políticas antes de transferirlas a sistemas reales. Además, en el ámbito de UAV, el aprendizaje por refuerzo ha mostrado resultados prometedores en tareas como navegación autónoma, evitación de obstáculos y seguimiento de trayectorias, donde la complejidad del entorno hace difícil aplicar métodos tradicionales de control [Kober et al., 2013].

2.2.4. Proximal Policy Optimization

Proximal Policy Optimization (PPO) es uno de los algoritmos de aprendizaje por refuerzo profundo más utilizados actualmente en problemas de control continuo. Fue propuesto como una alternativa que prioriza la estabilidad y la simplicidad operativa frente a los métodos de optimización de políticas precedentes, logrando un equilibrio sustancial entre la eficiencia computacional y el rendimiento del agente [Schulman et al., 2017].

La principal aportación de PPO surge de limitar el cambio que puede sufrir una política entre actualizaciones consecutivas, evitando modificaciones que sean muy diferentes entre sí y que puedan desestabilizar el proceso de aprendizaje. Esto se consigue mediante una función objetivo que restringe la magnitud de las actualizaciones.

Una de las principales ventajas de PPO es su facilidad de implementación y su robustez

frente a distintos tipos de entornos. Estas características han hecho que se convierta en uno de los algoritmos de referencia en robótica, simulación física y control de UAV.

En aplicaciones de control aéreo autónomo, PPO ha demostrado ser eficaz para aprender políticas de estabilización, seguimiento de trayectorias y adaptación frente a perturbaciones externas, lo que lo convierte en una opción adecuada para problemas donde el entorno cambia de forma continua y las acciones son de naturaleza continua [Koch et al., 2019].

Así, se ha escogido como algoritmo de entrenamiento PPO por ofrecer este mecanismo de limitación entre actualizaciones de la política que garantiza un aprendizaje progresivo sin colapsos catastróficos en dinámicas de vuelo complejas, generando comandos de control suaves que sean seguros y viables de transferir a un dron físico. Por otro lado, este es el único algoritmo que está plenamente optimizado en la plataforma Isaac Lab, que es la que se ha empleado para la elaboración de este trabajo.

No obstante, se plantea como trabajo futuro evaluar particularmente el algoritmo SAC al realizar una exploración basada en entropía que permitiría descubrir maniobras de persecución más ágiles, y por haber demostrado su viabilidad en la tarea de seguimiento de rutas en escenarios concretos [Huang et al., 2025]. Para ello sería necesaria su implementación y optimización en la plataforma Isaac Lab, resultando de gran interés realizar una comparativa con los resultados obtenidos en este trabajo.

2.3. Aprendizaje por refuerzo aplicado a UAV

El uso de algoritmos de aprendizaje por refuerzo en UAV ha experimentado un crecimiento exponencial debido a la capacidad de estos modelos para resolver problemas no lineales complejos donde los métodos de control clásicos presentan limitaciones.

2.3.1. Navegación autónoma

La navegación autónoma mediante aprendizaje por refuerzo se centra en la capacidad de un agente para tomar decisiones secuenciales de alto nivel en entornos dinámicos, incluyendo tareas como la planificación de trayectorias y la evitación de obstáculos. Este enfoque ha sido ampliamente estudiado en robótica móvil y aérea, donde la percepción del entorno se integra directamente en el proceso de toma de decisiones [Kober et al., 2013, Sutton et al., 1998].

A diferencia de los enfoques clásicos basados en planificación geométrica o métodos probabilísticos con modelos explícitos del entorno, el aprendizaje por refuerzo permite apren-

der políticas directamente a partir de observaciones sensoriales, como mapas de profundidad o información visual, sin requerir una representación completa del entorno. Esto resulta especialmente relevante en escenarios parcialmente observables o altamente dinámicos.

Un aspecto clave en este tipo de problemas es el diseño de la función de recompensa, que debe equilibrar objetivos como la progresión hacia la meta, la eficiencia de la trayectoria y la seguridad operativa. En este contexto, los métodos basados en gradiente de política han demostrado ser especialmente adecuados para espacios de estado de alta dimensionalidad y acciones continuas [Schulman et al., 2017].

2.3.2. Control de actitud

El control de actitud constituye la capa de bajo nivel en un sistema de control de UAV, encargada de regular la orientación del vehículo mediante los ángulos de roll, pitch y yaw. Tradicionalmente, este problema ha sido abordado mediante controladores clásicos como PID o LQR, que dependen de modelos dinámicos linealizados y suponen ciertas condiciones de operación [Mahony et al., 2012].

El aprendizaje por refuerzo ofrece una alternativa basada en datos, en la que el controlador se obtiene mediante interacción directa con el sistema, sin necesidad de una modelización explícita perfecta. Este enfoque ha mostrado potencial para mejorar la robustez frente a incertidumbres del modelo, perturbaciones externas y cambios en la dinámica del sistema [Kober et al., 2013].

En este contexto, el agente aprende políticas de control que minimizan el error angular y el esfuerzo de control, lo que permite obtener comportamientos más adaptativos en condiciones no ideales, como variaciones de carga o perturbaciones aerodinámicas. Esto resulta especialmente relevante en escenarios donde los supuestos de linealidad de los modelos clásicos no se cumplen.

2.3.3. Entrenamiento y simulación

El entrenamiento de agentes de aprendizaje por refuerzo en robótica aérea requiere una gran cantidad de interacciones con el entorno, lo que hace que la experimentación directa sobre hardware real sea poco práctica debido a su coste, riesgo y tiempo de ejecución. Por este motivo, la literatura emplea de forma generalizada entornos de simulación para el entrenamiento previo de políticas de control [Kober et al., 2013].

Un problema fundamental en este proceso es la denominada brecha simulación-real (Sim-to-Real gap), que aparece debido a las diferencias entre el modelo simulado y el com-

portamiento físico real del sistema. Estas discrepancias pueden afectar significativamente al rendimiento del controlador cuando se transfiere a hardware real.

Para mitigar este problema, se emplean técnicas como la aleatorización de dominio, en la que se introducen variaciones en parámetros físicos del sistema durante el entrenamiento, y la inyección de ruido en observaciones y acciones. Estas estrategias permiten mejorar la robustez de las políticas aprendidas y su capacidad de generalización a condiciones no vistas durante el entrenamiento [Tobin et al., 2017].

2.3.4. Evaluación de controladores

La evaluación de las políticas de control obtenidas mediante aprendizaje por refuerzo se fundamenta en un análisis multidimensional que vincula el rendimiento estadístico con las restricciones físicas del sistema. En la Tabla 2.3 se justifican las métricas empleadas, y las fórmulas aplicadas para su cálculo se detallan en las Tablas 4.1 y 4.4.

La integración de estas métricas en un análisis de radar permite visualizar el compromiso entre la agresividad del controlador y su estabilidad operativa. Este enfoque multiobjetivo es fundamental para identificar modelos que no solo maximicen la recompensa escalar, sino que presenten un perfil de vuelo compatible con las limitaciones físicas de los actuadores.

2.4. Simulador Isaac de NVIDIA

El desarrollo de algoritmos de control basados en aprendizaje por refuerzo para sistemas robóticos presenta la necesidad de interactuar masivamente con el entorno para que el agente aprenda políticas estables y robustas. En el caso de UAV, este proceso resulta especialmente costoso y arriesgado si se realiza directamente sobre hardware real, debido al desgaste mecánico, el consumo energético y la posibilidad de colisiones o pérdidas de estabilidad. Por este motivo, el entrenamiento en simulación se ha convertido en una práctica estándar, permitiendo realizar millones de iteraciones de entrenamiento en entornos controlados antes de la transferencia al sistema físico.

En este contexto, las plataformas de simulación de alta fidelidad han adquirido un papel central, ya que permiten modelar de forma realista tanto la dinámica física del vehículo como la interacción con sensores y perturbaciones externas. Dentro de estas herramientas, el ecosistema Isaac de NVIDIA ha ganado relevancia por combinar simulación física acelerada por GPU, renderizado fotorrealista y una integración directa con flujos de aprendizaje automático y aprendizaje por refuerzo [NVIDIA, 2026c].

Tabla 2.3: Dimensiones y métricas utilizadas para la evaluación del agente PPO.

Dimensión	Métrica	Justificación
Robustez	Robustness Score (robustness_pct)	Métrica basada en el coeficiente de variación del retorno durante evaluación ($CV = \sigma/\mu$). Valores altos indican comportamiento consistente y baja sensibilidad a perturbaciones o variaciones del entorno [Henderson et al., 2018], donde σ es la desviación estándar y μ la media aritmética.
Seguridad	Safety Score (success_pct)	Evalúa la ausencia de terminaciones críticas asociadas a pérdida de altitud, actitud insegura o activación del sistema de seguridad. Se prioriza el cumplimiento de restricciones físicas y operacionales [Ray et al., 2019].
Calidad del Control	Smoothness Score (smoothness_pct)	Mide la suavidad de las acciones de control mediante la penalización de variaciones bruscas entre comandos consecutivos ($-\Delta\text{Action}$). Fundamental para transferencia <i>Sim-to-Real</i> y reducción de fatiga mecánica [Hwangbo et al., 2019].
Seguimiento	Hovering Score (hovering_pct) Tracking Score (tracking_pct)	Evalúa la precisión en el seguimiento de trayectoria (<i>tracking</i>) o de mantenimiento en el aire (<i>hovering</i>) combinando errores de posición, orientación y velocidad lineal [Kober et al., 2013].
Estabilidad	Stability Score (stability_pct)	Mide la estabilidad dinámica del vehículo mediante recompensas asociadas a estabilidad actitudinal y reducción de velocidades angulares. Valores altos indican comportamiento estable y ausencia de oscilaciones [Lee et al., 2020].

2.4.1. Isaac Sim

Isaac Sim es una plataforma de simulación robótica desarrollada por NVIDIA sobre la infraestructura de Omniverse, diseñada para la creación de entornos físicos virtuales de alta fidelidad orientados al entrenamiento, validación y despliegue de sistemas robóticos [NVIDIA, 2026c]. Su principal característica es la integración del motor físico NVIDIA PhysX, que permite simular dinámicas rígidas, colisiones, articulaciones y sensores con un alto grado de realismo. La versión de Isaac Sim que se ha empleado en este proyecto es la 5.1.0.

Desde el punto de vista del aprendizaje por refuerzo, Isaac Sim proporciona una infraestructura adecuada para entrenar agentes en tareas complejas gracias a tres propiedades fundamentales: simulación paralela acelerada por GPU, generación de datos sintéticos y capacidad de incorporar sensores realistas como cámaras RGB, LiDAR e IMU [NVIDIA, 2026b]. Estas capacidades resultan especialmente útiles en UAV, donde la percepción y la interacción con entornos tridimensionales dinámicos son factores clave.

Además, Isaac Sim ha sido adoptado en múltiples líneas de investigación relacionadas con navegación autónoma, percepción visual y simulación de vehículos aéreos, consolidándose como una alternativa moderna frente a simuladores tradicionales como Gazebo o Air-Sim, especialmente en escenarios donde la fidelidad física y visual es crítica [Jacinto et al., 2024].

2.4.2. Isaac Lab

Isaac Lab constituye la evolución de los entornos de aprendizaje basados en Isaac Gym y Orbit, proporcionando un marco de trabajo orientado específicamente al entrenamiento de políticas de aprendizaje por refuerzo sobre Isaac Sim [NVIDIA, 2026a]. Su objetivo principal es facilitar la definición de tareas, recompensas, observaciones y acciones dentro de entornos altamente paralelizables.

A diferencia de Isaac Sim, cuyo enfoque principal es la simulación física y sensorial, Isaac Lab introduce una capa de abstracción centrada en el aprendizaje, permitiendo definir problemas de control de forma modular y escalable. Esto simplifica la implementación de algoritmos como PPO, SAC o A2C, integrando directamente el ciclo completo de entrenamiento y evaluación. La versión de Isaac Lab empleada en este proyecto es la 2.2.1.

Uno de los aspectos más relevantes de Isaac Lab es su capacidad para ejecutar miles de entornos en paralelo sobre GPU, reduciendo significativamente el tiempo de entrenamiento respecto a simuladores tradicionales. Esta capacidad resulta especialmente importante en UAV, donde la complejidad dinámica y la alta frecuencia de control requieren grandes volúmenes de experiencia para alcanzar políticas estables.

3. Planteamiento y definición de los problemas de control

3.1. Modelo de dron empleado

El sistema desarrollado utiliza un dron cuadricóptero simulado en el entorno físico de Isaac Lab, basado en un modelo dinámico de vehículo aéreo no tripulado (UAV) de cuatro rotores. El tipo de dron empleado, que se presenta en la Figura 3.1, ha sido seleccionado por su maniobrabilidad intrínseca y su representatividad dentro de las tareas de navegación autónoma enfocadas al aprendizaje por refuerzo [Mahony et al., 2012].

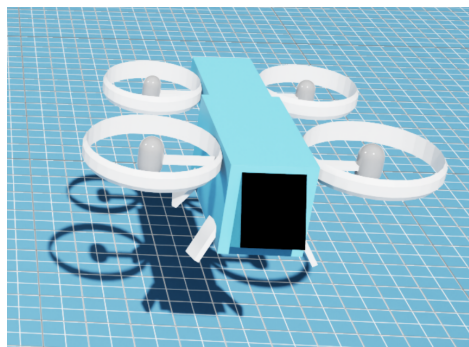


Figura 3.1: Modelo de dron empleado.

La aeronave opera con seis grados de libertad (6-DoF), correspondientes a las traslaciones en los ejes cartesianos x , y y z , así como a las rotaciones espaciales de alabeo (*roll*), cabeceo (*pitch*) y guiñada (*yaw*) [Luukkonen, 2011]. La traslación y orientación del vehículo

se generan exclusivamente a través de la variación de la velocidad angular y, en consecuencia, del empuje producido por cada uno de sus cuatro rotores independientes.

Debido a que el modelo de dron empleado posee rotores de distinto tamaño, y a que su distribución espacial no es simétrica respecto al centro de masas, la aplicación directa de las fórmulas descritas en la Tabla 2.2 no podrían ser aplicadas, y por lo tanto, surge la necesidad de emplear un modelo de Inteligencia Artificial en lugar de un control clásico, al permitir una gestión adecuada de las dinámicas complejas donde los modelos matemáticos tradicionales son muy complejos de ajustar. Así, el entorno simulado por Isaac Lab, por medio del motor de físicas PhysX, es el encargado de gestionar la dinámica no lineal acoplada de esta aeronave asimétrica.

3.2. Objetivos del dron

El propósito fundamental de este trabajo es lograr que el agente autónomo (el dron) aprenda una política de control capaz de resolver el problema de seguimiento de trayectorias dinámicas (*tracking*) mediante técnicas de aprendizaje por refuerzo. No obstante, debido a la elevada complejidad del control simultáneo de estabilidad, orientación y persecución de objetivos móviles en sistemas UAV, el proceso de aprendizaje se ha estructurado de forma progresiva en distintas etapas.

En una primera fase, el agente aprende el problema básico de estabilidad o vuelo estacionario (*hovering*) [Koch et al., 2019], donde el controlador debe contrarrestar la gravedad y las perturbaciones externas para mantener el dron estable alrededor de una posición fija minimizando el error posicional. Esta etapa inicial permite que la política aprenda los principios fundamentales de equilibrio dinámico antes de abordar comportamientos de navegación más complejos.

Así, los objetivos específicos asociados a esta primera fase de estabilización son:

1. Mantener el equilibrio dinámico del vuelo contrarrestando la gravedad sin generar oscilaciones perceptibles.
2. Minimizar el error de posición en los tres ejes respecto al punto objetivo marcado por el sistema.
3. Asegurar que las acciones mecánicas sobre los rotores sean suaves, evitando sobreesfuerzos estructurales que dificulten la transferencia a un dron real.
4. Sobrevivir durante todo el episodio de control sin infringir los límites de seguridad posicionales o de inclinación de la aeronave.

Con el objetivo de mejorar la estabilidad de la convergencia y simplificar el aprendizaje de la política, el entrenamiento se ha dividido en dos etapas principales:

- **Aprendizaje de estabilidad (*hover*).** En esta primera etapa el dron aprende a mantenerse estable alrededor de una posición fija. Esto permite ajustar la función de recompensa, validar el diseño del entorno y consolidar el control básico de la aeronave antes de introducir dinámicas más complejas.
- **Seguimiento de trayectoria (*tracking*).** Una vez obtenida una política capaz de estabilizar el dron de forma robusta, se incorporan objetivos dinámicos en el entorno para entrenar el seguimiento autónomo de rutas. En esta fase se modifican tanto la función de recompensa como el espacio de observaciones del agente, permitiendo que el dron aprenda a perseguir un objetivo móvil de manera continua y estable.

3.3. Definición del problema de estabilidad

3.3.1. Configuración del entorno

El entrenamiento se ha llevado a cabo utilizando Isaac Lab como plataforma base apoyada sobre el motor de físicas PhysX de NVIDIA [NVIDIA, 2026a, NVIDIA, 2026c]. Esta arquitectura destaca por su capacidad de paralelización masiva en GPU [Jacinto et al., 2024], permitiendo la instanciación simultánea de hasta 8196 entornos concurrentes. Esta escala resulta determinante para el aprendizaje por refuerzo, al permitir al agente acumular millones de transiciones en un tiempo muy reducido [Henderson et al., 2018].

En la simulación, cada entorno paralelo está compuesto por un suelo plano, la aeronave (cargada a través de un archivo USD o *Universal Scene Description*) y una esfera virtual de color rojo generada mediante un sistema de marcadores visuales. Esta esfera actúa como referencia gráfica del objetivo espacial tridimensional donde el cuadricóptero debe estabilizarse. Para propiciar un despegue controlado, el gestor de comandos del entorno inicializa la posición del objetivo exactamente en las mismas coordenadas espaciales donde aparece el dron en el primer instante de tiempo.

A nivel temporal, el motor físico computa las interacciones con un paso de simulación de $dt = 0.005$ s, equivalente a una frecuencia de 200 Hz. Sin embargo, la red neuronal no evalúa el estado a esta misma velocidad. Mediante un factor de reducción de 4, el controlador de IA actualiza sus acciones a 50 Hz. Esta disparidad permite que la física mantenga la alta resolución necesaria para modelar fuerzas giroscópicas o aerodinámicas complejas,

reduciendo al mismo tiempo la carga computacional del paso inferencial. Cada episodio se extiende hasta un máximo de 25 segundos, un horizonte suficiente para penalizar comportamientos divergentes a largo plazo. Con todas estas configuraciones, el aspecto del entorno para hover sería como el que se muestra en la Figura 3.2.

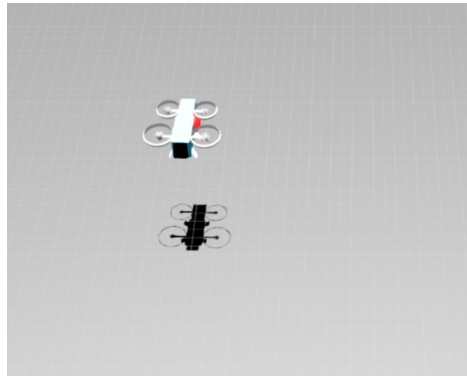


Figura 3.2: Visualización de un dron sobre su punto rojo de referencia.

3.3.2. Espacio de observaciones y acciones

El diseño del espacio de observaciones y acciones resulta crítico para la convergencia y el éxito de la política de control. En este trabajo se ha optado por un enfoque de tipo Actor-Crítico asimétrico [Konda and Tsitsiklis, 1999], lo que implica que la red del Actor (encargada de generar los comandos de control del dron) y la red del Crítico (responsable de evaluar la política durante el entrenamiento) procesan flujos de información distintos.

El espacio de observaciones define la percepción que el vehículo tiene del entorno. Para simular la imperfección inherente a los sensores reales y actuar simultáneamente como técnica de regularización, el Actor recibe una representación del estado sujeta a ruido gaussiano [Tobin et al., 2017, Hwangbo et al., 2019]. Por el contrario, el Crítico, al operar únicamente en simulación, tiene acceso a estados sin ruido. Esta asimetría permite calcular gradientes mucho más precisos y acelerar la convergencia del Actor, sin comprometer el realismo del despliegue final.

Por otro lado, el espacio de acciones determina cómo el agente interactúa con el entorno simulado. Las acciones generadas directamente por la red neuronal consisten en valores continuos normalizados en el intervalo $[-1, 1]$. Para transformar estas señales lógicas en comandos físicos viables y acelerar la convergencia del entrenamiento, el entorno calcula inicialmente la velocidad de rotación de vuelo estacionario (ω_{hover}) de forma individual para

cada dron. Este cálculo se realiza resolviendo un sistema de equilibrio matricial basado en la masa exacta del vehículo en ese episodio, gestionando así de forma adecuada las variaciones introducidas por la aleatorización del dominio.

A partir de este estado de equilibrio ideal, la acción dictada por la red neuronal modula la potencia final de cada rotor. Para mantener la estabilidad del aprendizaje, se emplea un factor de escala del 50%, de manera que el comando final se calcula como:

$$\omega = \omega_{\text{hover}} \cdot (1 + 0.5 \cdot a),$$

donde $a \in [-1, 1]$ es la acción del agente. Este diseño permite a la red incrementar o reducir la velocidad de cada motor en un $\pm 50\%$ respecto al punto en el que haría *hover*, garantizando que la red tenga el control suficiente para maniobrar sin inducir oscilaciones inmanejables al inicio del entrenamiento.

Para asegurar la integridad del motor de simulación física, se aplican funciones de saturación (*clipping*) en cascada. En primer lugar, se restringen los valores de ω para evitar velocidades de giro negativas. Posteriormente, durante la traducción a fuerzas aerodinámicas, el empuje vertical (Z) se limita a un máximo de 12000 N; este límite superior se estableció empíricamente tras observar que umbrales inferiores provocaban caídas sistemáticas y pérdida de control en episodios donde el dron presentaba inercias equivalentes a 2200 kg. Finalmente, el entorno inyecta de forma calculada las fuerzas de fricción lineal y los momentos torsores aerodinámicos (también limitados a un umbral de seguridad de ± 5000), garantizando así una alta fidelidad física en las dinámicas de vuelo.

El conjunto completo de variables que componen el estado observado se definen en la Tabla 3.2, y las acciones generadas y sus correspondientes restricciones físicas se detallan en la Tabla 3.1.

3.3.3. Configuración de PPO

El entrenamiento se ha llevado a cabo utilizando el algoritmo *Proximal Policy Optimization* (PPO) [Schulman et al., 2017], implementado a través de la librería RSL RL. Debido a la alta sensibilidad de este algoritmo a los hiperparámetros, se ha diseñado un *pipeline* automático basado en un algoritmo de búsqueda aleatoria de hiperparámetros automatizado (*Random Search*) [Bergstra and Bengio, 2012] para explorar el espacio de configuraciones de manera eficiente.

Este método consiste en muestrear combinaciones de valores de forma aleatoria a partir de distribuciones estadísticas o intervalos predefinidos para cada parámetro. A diferen-

Tabla 3.1: Definición del espacio de acciones y restricciones físicas del entorno.

Variable	Rango / Límite	Descripción y aplicación
Comandos de rotores	$[-1, 1]$	Salida directa de la política (4 valores continuos). Actúan como multiplicadores para modular la velocidad de giro de cada uno de los cuatro rotores.
Empuje aerodinámico	Máx. 12000 N	Fuerza de sustentación vertical (F_z) calculada a partir de las acciones. Se recorta numéricamente para evitar aceleraciones verticales imposibles.
Fricción aerodinámica	± 5000 N	Fuerza de resistencia o arrastre calculada en función de la velocidad del vehículo; aplicada en sentido contrario al movimiento.
Momentos torsores	± 5000 N·m	Pares de fuerza combinados derivados de la diferencia de empuje entre rotores opuestos y la fricción rotacional dinámica.

Tabla 3.2: Definición del espacio de observaciones, unidades y ruido aplicado.

Variable	Dimensión	Unidades	Descripción	Ruido Actor (σ)
Distancia relativa	3	m	Distancia en los ejes x, y, z desde la posición actual del dron hasta las coordenadas del punto objetivo.	0.01
Velocidad lineal	3	m/s	Componentes de la velocidad de traslación del vehículo proyectadas en su sistema de coordenadas local.	0.05
Velocidad angular	3	rad/s	Velocidad de rotación del dron en sus tres ejes espaciales (alabeo, cabeceo y guiñada).	0.05
Altura relativa	1	m	Distancia vertical (z) medida desde el centro de masas del vehículo hasta el origen del suelo de la simulación.	0.01
Gravedad proyectada	3	m/s^2	Vector de la aceleración gravitatoria descompuesto y proyectado sobre el sistema de referencia local del dron.	0.01

cia de las búsquedas exhaustivas tradicionales (como *Grid Search*), *Random Search* permite explorar un mayor número de valores únicos y descubrir configuraciones óptimas con un coste computacional significativamente menor, aprovechando que el rendimiento del modelo suele depender críticamente de solo un pequeño subconjunto de los hiperparámetros evaluados.

El proceso de búsqueda contempló un total de 35 configuraciones experimentales aleatorias. En cada experimento, los agentes se entrenaron durante 1001 iteraciones, procesando 100 pasos por entorno. El espacio de búsqueda incluyó variaciones en la arquitectura de las redes neuronales, probando capas ocultas desde [64, 64] hasta [256, 256], así como distintas funciones de activación (*ReLU*, *Tanh*, *ELU*, *GELU*).

Además, se exploraron rangos específicos para los parámetros fundamentales del aprendizaje: la tasa de aprendizaje varió entre 5×10^{-5} y 3×10^{-3} , el factor de descuento (γ) entre 0.95 y 0.99, y el coeficiente de entropía entre 0.0 y 0.05 para regular el balance entre exploración y explotación [Henderson et al., 2018]. Además, el sistema paraleliza el entrenamiento ajustando dinámicamente el número de entornos simultáneos, evaluando potencias de dos desde 64 hasta 8196 entornos concurrentes.

Así, la configuración empleada para el *Random Search* es la que se muestra en la Tabla 3.3.

Tabla 3.3: Espacio de búsqueda de hiperparámetros de PPO para el algoritmo Random Search.

Parámetro	Descripción	Valores explorados
init_noise_std	Desviación estándar del ruido gaussiano inicial añadido a las acciones continuas para fomentar la exploración térmica del entorno.	{0.1, 0.2, 0.5, 0.8, 1.0}
actor_hidden_dims	Arquitectura de la red neuronal del Actor, definida por el número de neuronas en cada capa oculta.	[64, 64], [128, 128], [256, 256], [128, 64]
critic_hidden_dims	Arquitectura de la red neuronal del Crítico, encargada de estimar la función de valor esperada.	[64, 64], [128, 128], [256, 256]

Tabla 3.3 – Continuación de la página anterior

Parámetro	Descripción	Valores explorados
activation	Función de activación no lineal aplicada en las capas ocultas de las redes neuronales.	ReLU, Tanh, ELU, GELU
value_loss_coef	Coefficiente que pondera la importancia de la función de pérdida del Crítico (error de valor) respecto a la pérdida total de la red.	{0.5, 1.0, 2.0}
clip_param	Umbral de recorte (ϵ) característico de PPO que limita drásticamente cuánto puede cambiar la política en una sola actualización.	{0.1, 0.2, 0.3}
entropy_coef	Factor de penalización que premia la entropía de la política para evitar la convergencia prematura a óptimos locales.	{0.0, 0.001, 0.01, 0.05}
num_learning_epochs	Número de pasadas de optimización sobre los datos recolectados en el búfer de experiencia antes de descartarlos.	{4, 8, 12}
num_mini_batches	Número de subconjuntos en los que se divide el lote de datos (<i>batch</i>) para realizar las actualizaciones de gradiente.	{2, 4, 8}
learning_rate	Tasa de aprendizaje inicial del optimizador (Adam), que determina el tamaño de actualización de los pesos.	$\{5 \cdot 10^{-5}, 10^{-4}, 3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}\}$
gamma	Factor de descuento (γ) en la ecuación de Bellman; determina la relevancia de las recompensas futuras frente a las inmediatas.	{0.95, 0.98, 0.99}
lam	Parámetro λ del estimador de Ventaja Generalizada (GAE) para equilibrar el sesgo y la varianza en las estimaciones.	{0.90, 0.95, 0.97}

Tabla 3.3 – Continuación de la página anterior

Parámetro	Descripción	Valores explorados
max_grad_norm	Valor máximo permitido para la norma de los gradientes (técnica de <i>clipping</i>) para evitar explosión de gradientes numéricos.	{0.5, 1.0, 2.0}
desired_kl	Divergencia Kullback-Leibler objetivo, utilizada como criterio de parada temprana temporal (<i>early stopping</i>) en las épocas.	{0.005, 0.008, 0.01, 0.02}

3.3.4. Estructura de recompensas y terminaciones

La formulación de la función de recompensa (*Reward shaping*) guía el comportamiento del dron para que logre estabilizarse [Sutton et al., 1998, Kober et al., 2013]. Se ha diseñado una función compuesta por múltiples términos ponderados, que se emplean teniendo en cuenta las métricas de evaluación que se presentan en la Tabla 2.3, y tras realizar una experimentación exhaustiva probando diversos términos adicionales, con diferente peso y ecuación de cálculo para cada término. Se puede consultar un resumen de los resultados obtenidos en la experimentación en los anexos A.2 y A.3.

Durante estas pruebas previas se evaluaron y descartaron varias estrategias alternativas que, aunque parecían intuitivas, no lograban una estabilización efectiva del vehículo. Para comprender mejor la complejidad del problema, a continuación se detallan los enfoques descartados y su efecto en el comportamiento del agente:

- **Tolerancias de vuelo estacionario (*hovering*):** Se intentó premiar al dron simplemente por mantenerse en el aire de un rango de altura aceptable, en lugar de exigirle una coordenada exacta. Sin embargo, esto provocaba que el agente se volviera conformista, oscilando constantemente dentro de esos márgenes de tolerancia sin llegar a estabilizarse por completo.
- **Rescate direccional de velocidad:** Consistía en dar puntos al vehículo si su velocidad apuntaba en la dirección correcta hacia el objetivo, sin importar a qué distancia estuviera. Esta estrategia fracasó porque incitaba al dron a acelerar de forma descontrolada para acumular más recompensa, lo que hacía que sobrepasara el destino sin lograr frenar a tiempo.

-
- **Penalizaciones cuadráticas directas:** Se probaron castigos matemáticos que crecían de forma muy agresiva cuanto más se alejaba el dron de su meta. El problema de este enfoque es que, ante errores o ráfagas iniciales grandes, el sistema recibía una penalización tan extrema que el dron reaccionaba con maniobras bruscas e inestables que solían terminar en colisión.
 - **Funciones de error "suaves" (*fine-grained* con tangentes hiperbólicas):** En contraposición al punto anterior, se utilizaron funciones que limitaban el castigo máximo cuando el dron estaba muy lejos. Aunque evitaba las reacciones bruscas, provocaba el efecto contrario: al estar lejos, el dron no veía la recompensa y por tanto no corregía su trayectoria, evitando cualquier movimiento y ralentizando enormemente el aprendizaje.

Tras analizar el impacto de estas alternativas, se determinó que el enfoque más robusto consistía en utilizar funciones exponenciales, que proporcionan correcciones firmes pero suaves. Así, los términos seleccionados son los que mostraron mejor rendimiento durante el entrenamiento:

1. **Estabilidad de actitud (Peso 8.0):** Recompensa el mantenimiento de una orientación plana, penalizando los ángulos de alabeo (ϕ) y cabeceo (θ) mediante la función exponencial $e^{-3.0(\phi^2 + \theta^2)}$.
2. **Mantenimiento de altitud (Peso 4.0):** Minimiza el error entre la altura actual z y la altura objetivo z_{target} mediante $e^{-2(z - z_{target})^2}$.
3. **Estabilidad de velocidad angular (Peso 4.0):** Penaliza los giros rápidos, dando un peso del 70% a las oscilaciones en los ejes XY (ω_{xy}), y un 30% a la velocidad de guiñada (ω_z). La fórmula aplicada es $0.7e^{-0.5\|\omega_{xy}\|^2} + 0.3e^{-1.5\omega_z^2}$.
4. **Precisión de posición (Peso 2.0):** Incentiva al dron a mantenerse lo más cerca posible de las coordenadas XY del objetivo ($p_{target_{xy}}$), calculándose mediante $e^{-0.5\|p_{xy} - p_{target_{xy}}\|^2}$, donde p_{xy} es la posición actual del dron en las coordenadas XY.
5. **Suavidad de acciones (Peso 3.0):** Penaliza las variaciones bruscas entre la acción actual a_t y la del instante anterior a_{t-1} con la expresión $(-0.01\|a_t - a_{t-1}\|^2)$. Esto reduce las vibraciones y el desgaste mecánico simulado (necesario para el paso de *Sim-to-Real*).
6. **Penalización de velocidad (Peso 1.5):** Castiga velocidades lineales (v) muy elevadas para promover un vuelo más conservador y estable, aplicando la función $e^{-1.0\|v\|^2}$.

Condiciones de terminación: Para evitar que el agente explore estados irrecuperables o físicamente imposibles, los episodios finalizan anticipadamente bajo las siguientes condiciones:

- **Inclinación crítica:** Si el alabeo o cabeceo superan los 60° ($\pi/3$ radianes). Dado que la aeronave está concebida para el transporte de pasajeros o mercancía, esta restricción es indispensable para garantizar la seguridad del viaje, penalizando cualquier maniobra agresiva que ponga en riesgo a los ocupantes o a la carga.
- **Pérdida de altitud:** Si el centro de masas cae por debajo del límite estructural de la aeronave (5.06 m del tren de aterrizaje + 0.05 m de margen). Debido a que el entorno simulado no implementa dinámicas explícitas de colisión, rebasar este umbral inferior se asume como un impacto directo contra el suelo, lo que hace irre recuperable el estado del vehículo.
- **Protección del simulador:** Se reinicia el entorno si la red genera valores nulos (NaNs), si la aeronave alcanza velocidades irreales superiores a 100 m/s, o si sale despedida a más de 200 metros del origen. Esto no solo previene divergencias matemáticas en el motor físico, sino que asegura que cada agente se mantenga dentro de un cubo de 200 x 200 metros, que es el espacio que se ha dejado entre cada entorno.

Estas terminaciones tempranas aceleran significativamente la convergencia del entrenamiento al evitar que el agente desperdicie tiempo computacional explorando trayectorias inválidas. Además, garantizan que el comportamiento aprendido se mantenga dentro de los límites operativos seguros, facilitando una futura transferencia de la política a una plataforma física real.

3.3.5. Técnicas de prevención de sobreajuste

El sobreajuste (*overfitting*) es un fenómeno que ocurre cuando un modelo de aprendizaje automático se adapta de manera excesiva a su entorno de entrenamiento, memorizando situaciones específicas y ruido en lugar de aprender comportamientos o reglas generales. En el ámbito del aprendizaje por refuerzo, un agente sobreajustado exhibirá un rendimiento excelente en las condiciones exactas en las que fue simulado, pero será incapaz de reaccionar adecuadamente ante escenarios nuevos, dinámicas imprevistas o durante su transferencia al mundo real (*sim-to-real*).

La capacidad de un agente autónomo para ignorar el ruido operativo del entorno y generalizar sus actuaciones depende tanto de las variaciones físicas que observe como de las

restricciones algorítmicas impuestas durante su entrenamiento. Para garantizar la robustez de la política y evitar la memorización de trayectorias, se han combinado estrategias de aleatorización del dominio con mecanismos de regularización durante la optimización matemática.

En primer lugar, el sistema inyecta aleatoriedad en las propiedades físicas y en las condiciones iniciales de cada episodio. Esta es una técnica conocida en aprendizaje por refuerzo como *Domain Randomization* [Tobin et al., 2017]. Cada vez que el dron debe reaparecer, su posición espacial se escoge mediante una distribución uniforme en un prisma de $10 \times 10 \times 4$ metros ($x, y \in [-5, 5], z \in [8, 12]$). Por otro lado, el cuadricóptero inicia el episodio con rotaciones impredecibles en todos sus ejes y velocidades residuales que lo empujan en direcciones aleatorias de hasta 0.5 m/s. Para dotar de resistencia aerodinámica e inercial a la política resultante, cada instancia del entorno modifica dinámicamente la masa del vehículo un $\pm 10\%$ respecto a su valor nominal, simulando diferentes estados de carga o variaciones estructurales. En paralelo, se inyectan fuerzas que actúan a modo de ráfagas de viento instantáneas sobre la plataforma, con componentes de velocidad variables entre $[-1, 1]$ m/s en el plano transversal y $[-0.5, 0.5]$ m/s en el plano vertical.

En segundo lugar, se ha implementado un mecanismo de parada temprana (*early stopping*) para detener el entrenamiento cuando el modelo entra en una fase de estancamiento, evitando así el sobreajuste a las dinámicas concretas de los últimos episodios experimentales [Henderson et al., 2018]. Tras evaluar múltiples configuraciones para este mecanismo, se determinó como configuración final una paciencia (número máximo de iteraciones sin mejora) de 500 iteraciones con un umbral de mejora mínima de 0. Bajo esta parametrización, el proceso de optimización se interrumpe de forma automática únicamente si el agente es incapaz de registrar cualquier mejora marginal en su función de retorno durante esas 500 iteraciones consecutivas.

Por último, cabe destacar que durante la fase de diseño algorítmico se evaluó la viabilidad de integrar otras técnicas clásicas de regularización del aprendizaje profundo. Concretamente, se probó la aplicación de apagado neuronal (*dropout*) en las capas ocultas de las redes, así como estrategias de decaimiento dinámico (*decay*) tanto para el coeficiente de entropía como para el umbral de recorte ϵ (*clip param*) característico de PPO. No obstante, los experimentos preliminares evidenciaron que la inclusión de estos mecanismos no aportaba ningún beneficio a la convergencia. Por el contrario, introducían una pérdida severa de estabilidad en el aprendizaje y una degradación notable en la calidad del vuelo estacionario, motivo por el cual fueron descartados de la arquitectura definitiva.

A modo de aclaración, las técnicas de parada temprana, *dropout*, y el decaimiento di-

námico fueron implementadas modificando el código base de la plataforma de Isaac Lab, puesto que no venían implementadas por defecto.

3.4. Definición del problema de seguimiento de rutas

Una vez establecida y validada la capacidad del cuadricóptero para mantener el equilibrio dinámico en un punto estático, el problema evoluciona hacia el seguimiento de trayectorias dinámicas (*tracking*) [Farid et al., 2018]. En este escenario, el objetivo principal ya no es permanecer inmóvil, sino perseguir a un objetivo móvil a lo largo del espacio tridimensional, igualando su velocidad y orientación [Park et al., 2021]. De este modo, se emulan miles de planes de vuelo aleatorios en paralelo que permitan un entrenamiento óptimo de los drones con aprendizaje por refuerzo, evitando el sobreajuste a rutas específicas.

3.4.1. Configuración del entorno

La arquitectura general de simulación (motor PhysX, frecuencias de integración física a 200 Hz y actualización de la política a 50 Hz, así como el horizonte del episodio de 25 segundos) se mantiene idéntica a la descrita en el problema de estabilidad. Sin embargo, la lógica de simulación interna experimenta modificaciones significativas para dar soporte al comportamiento dinámico.

En primer lugar, el marcador objetivo (la esfera roja) deja de ser estático. El sistema le asigna una velocidad de desplazamiento aleatoria inicial comprendida entre 3 y 6 m/s, con variaciones dinámicas en su dirección (curvatura de trayectoria) y alteraciones verticales estocásticas durante el vuelo. Para garantizar que la persecución no exceda los límites lógicos del simulador ni derive en inestabilidades numéricas, se ha implementado un ortoedro virtual. Este mecanismo confina el movimiento del objetivo dentro de un volumen operativo de 90×90 metros en el plano horizontal y entre 1.75 y 20.0 metros en el vertical; si el objetivo alcanza estas fronteras, su velocidad se invierte automáticamente simulando un rebote aerodinámico.

Por otro lado, se han incorporado nuevos elementos visuales al entorno para facilitar el análisis cualitativo de la orientación durante la persecución. Se proyecta un vector direccional de color verde que indica la trayectoria instantánea del punto objetivo, y un vector rojo anclado a la parte delantera del dron que señala su orientación actual (*heading*). Estos marcadores resultan fundamentales para confirmar visualmente que el agente no solo persigue al objetivo, sino que lo hace orientándose hacia la dirección del movimiento. Con

todas estas configuraciones, el aspecto del entorno para el seguimiento de rutas sería como el que se muestra en la Figura 3.3.

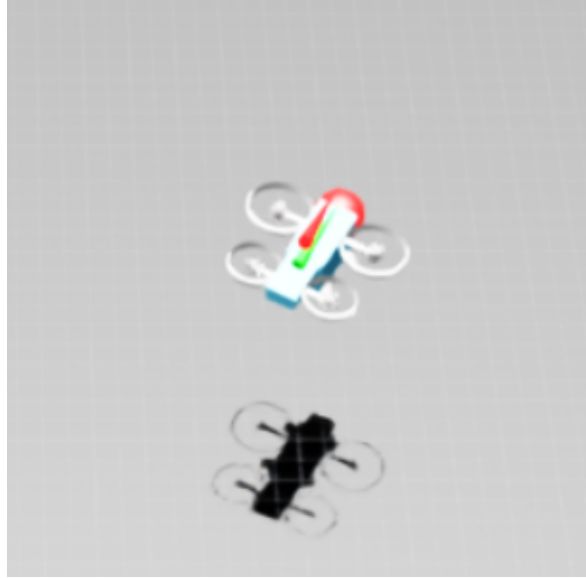


Figura 3.3: Visualización de un dron siguiendo su punto rojo de referencia con los marcadores de dirección del dron y el ideal.

3.4.2. Espacio de observaciones y acciones

A diferencia de los enfoques de seguimiento temporales clásicos (como el propuesto en [Casado et al., 2026]), donde el plan de vuelo genera un vector intermedio de velocidad de comando (v_{cmd}) como paso previo al control de bajo nivel, en este trabajo se propone un *pipeline* directo. El agente de aprendizaje por refuerzo procesa como entrada la posición predictiva futura y el estado actual para comandar directamente las velocidades angulares de los rotores, delegando la compensación implícita del error cinemático en la propia política de PPO.

El espacio de acciones y la transformación matemática que convierte las salidas continuas de la red en empuje aerodinámico y momentos torsores se mantiene inalterado respecto a la Tabla 3.1 del problema estático. No obstante, este nuevo enfoque del problema exige una transformación sustancial en el espacio de observaciones.

Incluir únicamente la distancia relativa al objetivo resulta insuficiente para un seguimiento a altas velocidades, ya que no aporta información predictiva. Por ello, se introducen nuevas variables cinemáticas calculadas desde el sistema de coordenadas local del dron. Por lo

tanto, el vector de estado ya no contiene la posición absoluta y pasa a incorporar la velocidad lineal del objetivo, el error de orientación (*yaw*) respecto a la trayectoria, y una variable predictiva. Esta última calcula dónde se encontrará el objetivo en un horizonte temporal futuro (entre 0.4 y 0.9 segundos de anticipación) asumiendo una velocidad constante, lo que otorga al agente la capacidad de anticiparse a las curvas [Park et al., 2021].

El modelo Actor-Crítico sigue aplicando ruido de forma asimétrica (solo en el Actor) para emular la incertidumbre sensorial. El nuevo espacio de observaciones configurado para esta fase se describe en la Tabla 3.4.

Tabla 3.4: Espacio de observaciones, unidades y ruido aplicado para el problema de seguimiento de rutas.

Variable	Dimensión	Unidades	Descripción	Ruido Actor (σ)
Velocidad lineal	3	m/s	Velocidad de traslación actual del dron.	0.05
Velocidad angular	3	rad/s	Velocidad de rotación del dron en sus ejes locales.	0.05
Gravedad proyectada	3	m/s ²	Vector gravitatorio proyectado en el sistema del dron.	0.01
Velocidad del objetivo	3	m/s	Vector de velocidad del punto guía visto desde el dron.	0.01
Posición predictiva futura	3	m	Vector direccional hacia la coordenada donde estará el objetivo en un horizonte de anticipación simulado.	0.01
Error de guiñada (<i>yaw</i>)	1	rad	Diferencia angular normalizada entre la orientación actual del dron y la dirección de movimiento del objetivo.	0.01

3.4.3. Configuración de PPO

El marco general de optimización mantiene la exploración de hiperparámetros (*Random Search*) detallada previamente. No obstante, las exigencias de la convergencia en un entorno altamente dinámico han motivado dos ajustes empíricos en la parametrización de PPO.

En primer lugar, el horizonte temporal analizado en cada actualización de la red se ha

incrementado de 100 a 128 pasos. Esta expansión de la ventana temporal permite a la red capturar secuencias dinámicas más prolongadas, algo fundamental cuando se evalúan inercias en curvas a alta velocidad. En segundo lugar, se ha habilitado la normalización empírica de observaciones. Dado que las velocidades y los vectores predictivos alcanzan magnitudes numéricas muy superiores a las presentes en el vuelo estático, esta capa de normalización automática previene la saturación inicial de las funciones de activación, estabilizando los gradientes en las primeras fases del entrenamiento. Por último, para maximizar el flujo de experiencia, se ha fijado el tamaño del lote a 8196 entornos paralelos continuos.

3.4.4. Estructura de recompensas y terminaciones

Las restricciones físicas de terminación del episodio (caída libre, inclinación superior a 60° o explosión numérica) se mantienen intactas para asegurar que el modelo siga siendo físicamente plausible. Sin embargo, tras llevar a cabo diversos experimentos evaluando diferentes enfoques de vuelo, la función de recompensa ha sufrido una reestructuración profunda, priorizando ahora el seguimiento de un punto móvil frente a la estabilidad necesaria previa.

En esta nueva etapa se conservan algunos términos del entrenamiento de *hovering* (como la estabilidad de actitud, la velocidad angular y la suavidad de las acciones), ya que siguen siendo vitales para evitar comportamientos inestables. No obstante, sus pesos se reducen drásticamente: la penalización de actitud cae de 8.0 a 2.0 empleando una curva de decaimiento más permisiva, y la penalización por acciones bruscas se ajusta a un mínimo de 0.1, permitiendo a los motores reaccionar agresivamente ante cambios de trayectoria.

Por el contrario, la recompensa de mantenimiento de altitud y la penalización global de velocidad lineal desaparecen por completo. Esto se debe a que, para que un cuadricóptero alcance objetivos móviles a altas velocidades (como 6 m/s), es físicamente indispensable que rompa su perfil de vuelo plano y se incline (*pitch* o *roll*) para redirigir el vector de empuje, necesitando total libertad para adaptar su altura y velocidad a la persecución del objetivo.

El objetivo principal de la optimización pasa entonces a centrarse en tres nuevos términos cinemáticos orientados al seguimiento dinámico:

1. **Alineación de velocidad (Peso 8.0):** Actúa como el motor principal del aprendizaje. Se calcula mediante el producto escalar entre el vector normalizado de la velocidad actual del dron ($\hat{v}$) y el de la velocidad del objetivo ($\hat{v}_{target}$). Premia directamente volar en la misma dirección y sentido que la referencia, acotando el resultado matemático entre 0 y 1 mediante la fórmula $\text{clamp}(\hat{v} \cdot \hat{v}_{target}, 0, 1)$.

2. **Alineación del frontal (Peso 3.0):** Fomenta que el dron mire hacia donde vuela, evaluando el producto escalar entre el vector de dirección frontal del morro en el plano horizontal (definido a partir de su ángulo de guiñada ψ , como $\vec{f} = (\cos \psi, \sin \psi)$) y su propio vector de velocidad normalizado en dicho plano ($\hat{v}_{xy}$). La expresión aplicada es $\text{clamp}(\vec{f} \cdot \hat{v}_{xy}, 0, 1)$.
3. **Precisión de posición (Peso 2.0):** Este término es una modificación al empleado en el problema de estabilidad. Mantiene la persecución cercana del objetivo utilizando una formulación más permisiva que la empleada durante el vuelo estacionario. La recompensa se calcula directamente mediante la expresión $e^{-0.05\|p-p_{target}\|^2}$. El coeficiente reducido permite conservar gradientes útiles incluso cuando el objetivo se encuentra alejado, evitando que la señal de aprendizaje desaparezca durante persecuciones rápidas. En este caso, p y p_{target} representan las coordenadas del dron y del objetivo respectivamente en los ejes XYZ.

Así, la Tabla 3.5 resume las diferencias principales entre la función de recompensa utilizada durante el aprendizaje de estabilidad y la empleada posteriormente para el seguimiento de trayectorias.

Tabla 3.5: Comparación entre recompensas de hovering y seguimiento de trayectoria.

Término de recompensa	Hovering	Tracking	Estado
Estabilidad de actitud	8.0	2.0	Reducido
Estabilidad de velocidad angular	4.0	1.0	Reducido
Suavidad de acciones	3.0	0.1	Reducido
Mantenimiento de altitud	4.0	—	Eliminado
Penalización de velocidad	1.5	—	Eliminado
Precisión de posición	2.0	2.0	Modificado
Alineación de velocidad	—	8.0	Nuevo
Alineación del frontal	—	3.0	Nuevo

3.4.5. Técnicas de prevención del sobreajuste

Las estrategias metodológicas de parada temprana (*early stopping* con paciencia de 500 iteraciones) y aleatorización de dominio (variaciones de masa y viento) operan bajo las mis-

mas directrices explicadas en el control estático.

Como aportación singular a la fase de seguimiento, la variable de anticipación temporal empleada para calcular la observación predictiva actúa como una técnica de regularización algorítmica adicional. El tiempo de predicción no es constante, sino que se muestrea aleatoriamente para cada instancia paralela dentro del rango de 0.4 a 0.9 segundos. Al forzar al agente a lidiar continuamente con proyecciones más largas o más cortas según el episodio, se evita que la red neuronal memorice una distancia de retraso específica, desarrollando en su lugar una verdadera comprensión geométrica de la trayectoria.

4. Experimentación y resultados

En este capítulo se detalla la metodología empleada, la infraestructura de simulación y el análisis exhaustivo de los resultados del entrenamiento del dron, estructurados en dos fases secuenciales (control de estabilidad estacionaria y seguimiento dinámico de trayectorias) debido a la alta dimensionalidad del espacio aerodinámico y al elevado coste computacional. Para gestionar esta complejidad, se ha diseñado un flujo de trabajo global automatizado en la plataforma Isaac Lab (ilustrado en la Figura 4.1) que abarca desde la configuración inicial del entorno y las recompensas, hasta la generación estocástica de hiperparámetros, la simulación paralela y la evaluación estadística final. Este ciclo completo, que requiere aproximadamente 70 horas de cómputo ininterrumpido, garantiza una exploración sistemática y reproducible del espacio de soluciones algorítmicas.

4.1. Experimentos definidos para la estabilidad

El objetivo de esta fase es identificar la parametrización exacta del algoritmo *Proximal Policy Optimization* (PPO) que maximice la precisión de la posición del dron minimizando simultáneamente las oscilaciones que se puedan producir.

Como la configuración manual de hiperparámetros en algoritmos de aprendizaje por refuerzo profundo resulta ineficiente y susceptible a sesgos, se ha implementado un proceso automatizado basado en la técnica de búsqueda aleatoria (*Random Search*), que es un método de optimización de hiperparámetros que prueba combinaciones de valores elegidos al azar para encontrar una configuración óptima [Bergstra and Bengio, 2012]. Para ello se ha definido un *pipeline* dividido en dos bloques secuenciales: generación y entrenamiento, seguidos de evaluación y análisis estadístico.

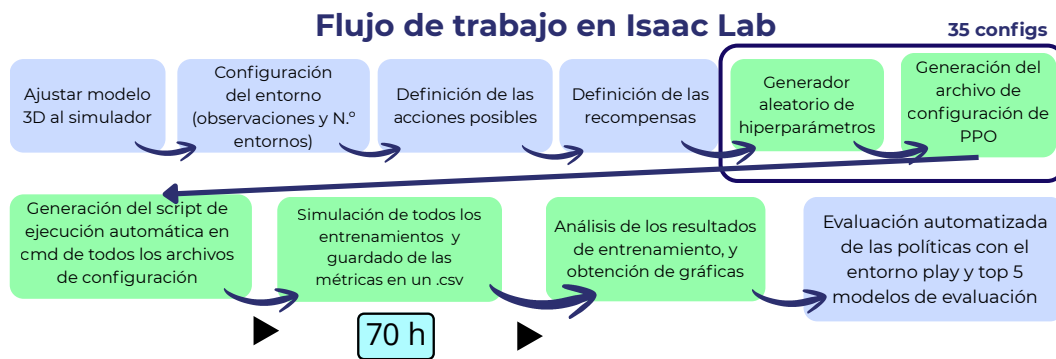


Figura 4.1: Diagrama global del flujo de trabajo automatizado diseñado para la experimentación en Isaac Lab.

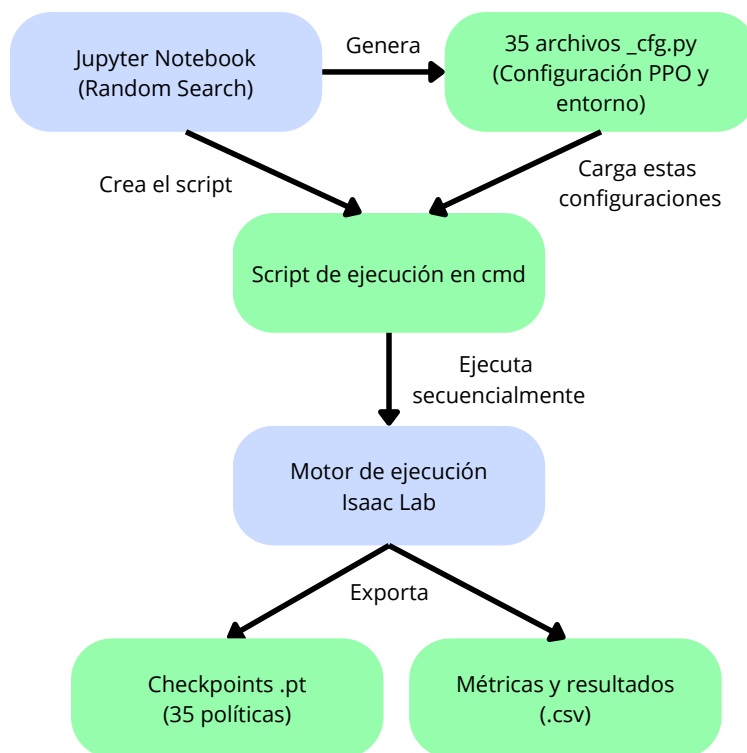


Figura 4.2: Arquitectura del flujo de trabajo para la generación paramétrica y ejecución secuencial de los entrenamientos.

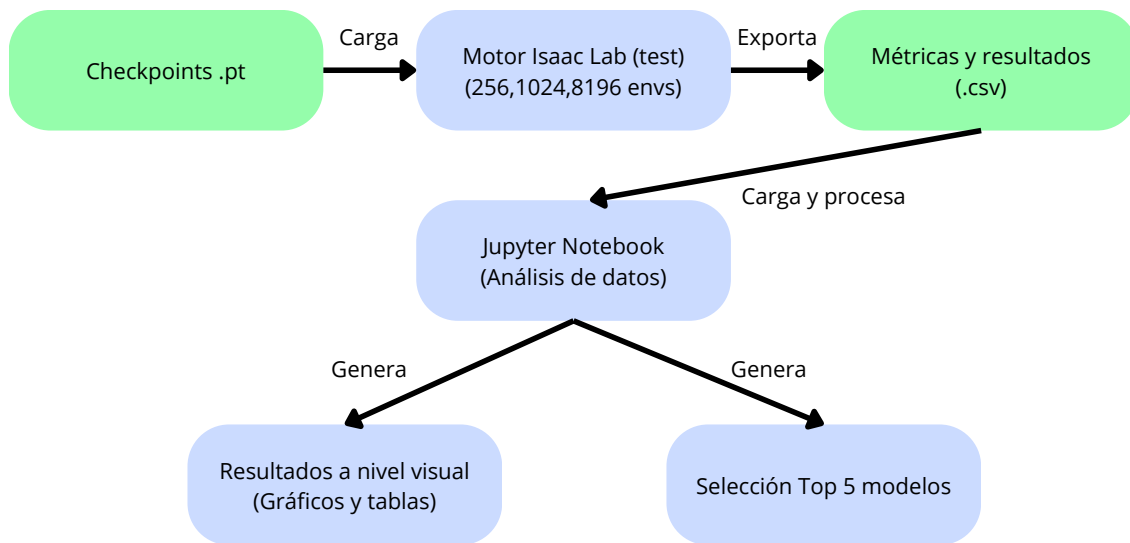


Figura 4.3: Flujo de procesamiento de datos para la evaluación masiva, extracción de métricas escaladas y selección estadística del modelo óptimo.

En la etapa de generación, una libreta de *Jupyter* actúa como núcleo del sistema, tal y como se detalla en la Figura 4.2. Así, muestrea estocásticamente 35 configuraciones distintas a partir del espacio de hiperparámetros, de forma que, para cada iteración, el sistema genera un archivo de código independiente que sobrescribe las variables específicas del experimento, tales como la topología de la red neuronal, la tasa de aprendizaje inicial, el umbral de recorte de la política y el factor de descuento.

Una vez finalizada la fase de entrenamiento, el sistema procede a la etapa de evaluación y análisis de resultados, cuyo flujo de ejecución se expone en la Figura 4.3. Con el propósito de comparar el desempeño y establecer un *ranking* objetivo entre los distintos agentes, se ha formulado una métrica global propia, normalizada en una escala porcentual de 0 a 100. Esta calificación se obtiene mediante la suma ponderada de las cinco dimensiones de evaluación previamente definidas en la Tabla 2.3, diseñadas para cuantificar la viabilidad del modelo siguiendo una estructuración jerárquica de los objetivos de control fundamentada en la literatura de robótica aérea autónoma.

Así, se otorga la mayor ponderación a la consecución de la tarea principal, que es el Seguimiento (35%), después le siguen las restricciones críticas de vuelo, representadas por la Seguridad (25%) y la Estabilidad (20%), indispensables para mantener al vehículo en el aire y evitar estados terminales prematuros [Hwangbo et al., 2017]; mientras que la Robustez

(10%) y la Calidad del Control (10%) actúan como términos de regularización para penalizar oscilaciones y garantizar que los comandos generados sean físicamente viables en un dron real (*sim-to-real*) [Kaufmann et al., 2020]. La composición exacta de estas dimensiones, los términos formales del entorno empleados para la tarea de *hovering* y el desglose de las operaciones matemáticas aplicadas se detallan en la Tabla 4.1.

Tabla 4.1: Cálculo de las métricas en Hovering.

Dimensión Normalización	Fórmula Aplicada	Símbolo	Término evaluado	Límite (clip)	Rango Real
Seguimiento (35%) Rango: [0.0, 5.5]	$\mathcal{L}(\text{clip}(r_{alt})) + \mathcal{L}(\text{clip}(r_{pos})) + \mathcal{L}(\text{clip}(r_{vel}))$	r_{alt}	Mantenimiento de altitud	≤ 3.0	[0, 4.0]
		r_{pos}	Precisión de posición	≤ 1.5	[0, 2.0]
		r_{vel}	Penalización de velocidad	≤ 1.0	[0, 1.5]
Seguridad (25%) Rango: [0.0, 1.0]	$1 - (t_{alt} + t_{att} + t_{safe})$	t_{alt}	Pérdida de altitud	—	[0, 1]
		t_{att}	Inclinación crítica	—	[0, 1]
		t_{safe}	Protección del simulador	—	[0, 1]
Estabilidad (20%) Rango: [0.0, 12.0]	$r_{att_stab} + r_{ang_stab}$	r_{att_stab}	Estabilidad de actitud	—	[0, 8.0]
		r_{ang_stab}	Estabilidad velocidad angular	—	[0, 4.0]
Robustez (10%) Rango: [0.0, 1.0]	$CV = \frac{\sigma}{\mu}$	CV	Coeficiente de variación	—	[0, 1.0]
Calidad Control (10%) Rango: [0.0, 0.048]	$-r_{act}$	r_{act}	Suavidad de acciones	—	[-0.048, 0]

En la Tabla 4.1, los símbolos que comienzan por ' r —' corresponden a las recompensas, mientras que los iniciados con ' t —' representan las tasas de terminación. El operador clip

satura las recompensas para establecer un límite realista teniendo en cuenta que se trata de un dron de 5 metros de largo y 2000 Kg de masa, y que por tanto, la perfección de posición en un punto, velocidad o altura se considerarán alcanzadas aunque haya cierta desviación. Por otro lado, el operador de linealización ($\mathcal{L}(x)$) se aplica a las funciones de recompensa que se calculan de forma exponencial para linealizar el error. Finalmente, para poder calcular la puntuación final del agente el resultado de cada fórmula se normaliza al rango general $[0, 1]$ y se multiplica por su peso final indicado con un '%’.

Para garantizar la fiabilidad estadística de estas pruebas de evaluación, todos los modelos se someten a tres ejecuciones de prueba independientes, variando el volumen de instanciación del simulador a 256, 1024 y 8196 entornos paralelos.

4.2. Resultados de la experimentación para la estabilidad

Tras ejecutar la fase de entrenamiento mediante búsqueda aleatoria (*Random Search*), se procedió a realizar un análisis exhaustivo del rendimiento de las distintas configuraciones. El primer paso consistió en evaluar la convergencia general del algoritmo PPO en el entorno simulado.

Como se observa en el análisis de la recompensa media de la Figura 4.4, la exploración de hiperparámetros logró hallar múltiples configuraciones capaces de superar holgadamente la barrera de los 350 puntos de recompensa. Esto confirma que el diseño del entorno y las funciones de recompensa establecen un gradiente de aprendizaje sólido y convergente para la tarea de vuelo estacionario.

A modo representativo, se puede observar en la Figura 4.5 la evolución de la recompensa durante la fase de entrenamiento de los mejores cinco modelos que se seleccionaron posteriormente en la fase final de validación. En los 5 casos se puede apreciar un ascenso pronunciado durante las primeras 200 iteraciones, alcanzando un régimen estacionario de alto rendimiento. Destaca la configuración 21 (color amarillo), que presenta una convergencia más tardía pero logra alcanzar una recompensa parecida a la del resto de modelos aproximadamente en la iteración 500.

No obstante, al transferir estas políticas pre-entrenadas al entorno de validación, se hace evidente la presencia de sobreajuste. La Figura 4.6 ilustra una caída generalizada en la recompensa al transferir al agente del entrenamiento a las pruebas de 256, 1024 y 8196 entornos.

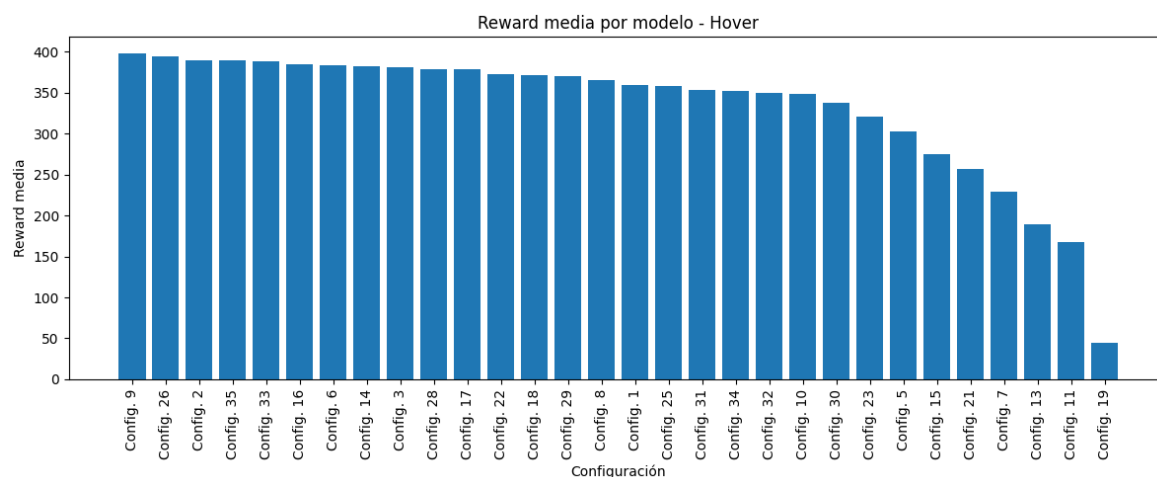


Figura 4.4: Distribución de la recompensa media obtenida por cada uno de los 35 modelos durante la fase de entrenamiento.

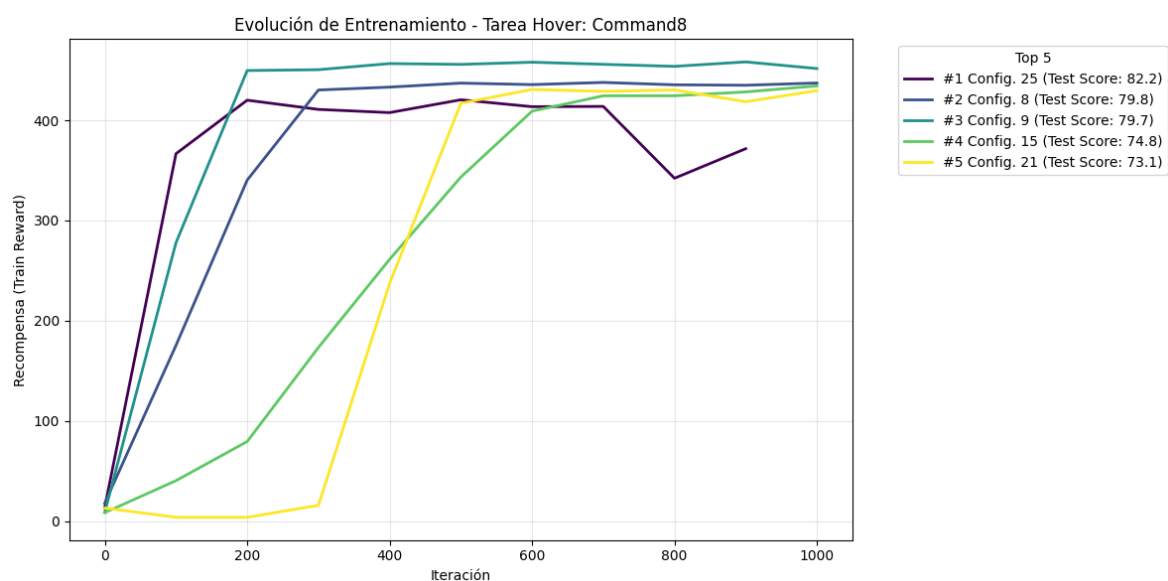


Figura 4.5: Curvas de evolución temporal de la recompensa durante el entrenamiento, filtradas exclusivamente para los cinco mejores modelos resultantes de la fase de validación (Test).

A pesar de las técnicas de prevención de sobreajuste aplicadas, este ha sido el mejor valor obtenido tras una extensa experimentación. Sin embargo, también cabe remarcar que esta caída no es pareja a la representación física real de la política empleada por los drones, ya que diversos términos empleados en la recompensa emplean funciones exponenciales,

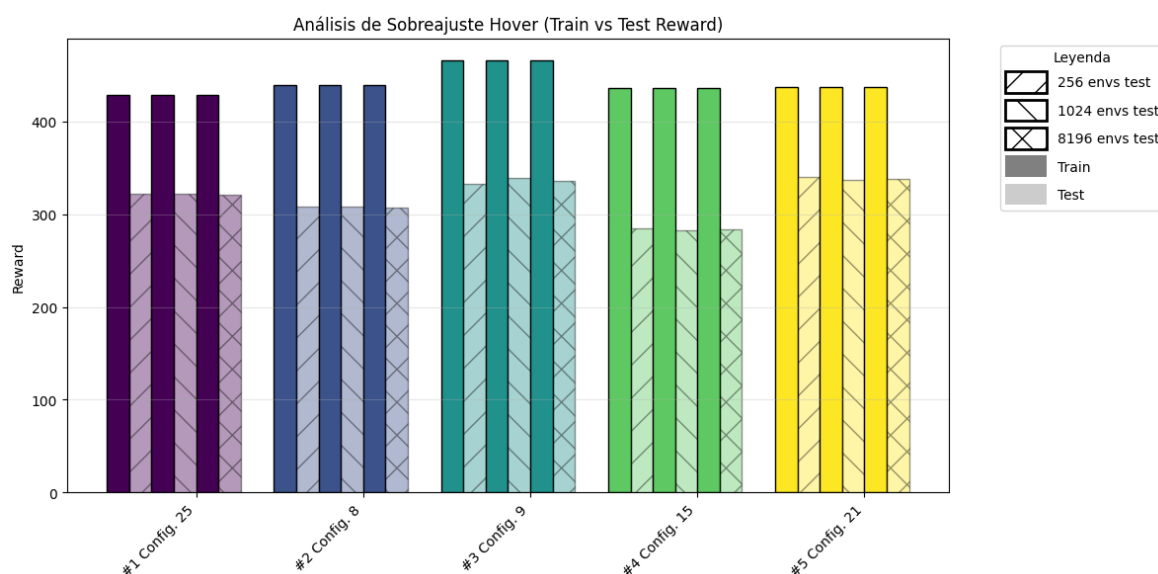


Figura 4.6: Análisis del sobreajuste (*overfitting*): comparativa entre la recompensa bruta de entrenamiento (barras sólidas) y el rendimiento en los entornos de evaluación paralelos (barras rayadas).

especialmente en la posición, lo que implica que la penalización en la recompensa es exponencial y no lineal. Es por esto que la comparativa de modelos no se realiza directamente en base a esta recompensa resultante, si no a una ponderación de cada término individual en base al tipo de fórmula empleado para su cálculo, con el objetivo de que todos los términos representen la degradación lineal de la política y no exponencial, como se puede observar en la Tabla 4.1.

Así, la Tabla 4.2 expone el desglose de esta puntuación para los 5 mejores modelos obtenidos. La configuración 25 es la que mayor puntuación presenta, con un 82.2% de puntuación global. Es notable observar cómo, a pesar de sufrir una caída del 25% en su recompensa, logra mantener niveles excelentes en suavidad de control (97.44%) y robustez (92.55%), variables críticas para un entorno físico real.

Para comprender el origen físico y algorítmico de este éxito y de las sutiles variaciones de rendimiento entre los agentes, resulta esencial analizar su arquitectura interna. La Tabla 4.3 expone la configuración exacta de hiperparámetros y estructura de redes neuronales (Actor-Crítico) que la búsqueda aleatoria asignó a estos cinco modelos. Las diferencias en la dimensionalidad de las capas ocultas y el ruido de exploración inicial (σ) justifican las distintas velocidades de convergencia observadas previamente en la Figura 4.5.

El perfil de comportamiento de estas políticas y su compromiso operativo se comprende visualmente a través de los diagramas de radar (Figura 4.7).

Tabla 4.2: Resultados globales de la evaluación multidimensional para los 5 mejores modelos de vuelo estacionario, ordenados por su *Score Global* en Test.

Modelo	Score Train (%)	Score Test (%)	Hover Test (%)	Seguridad Test (%)	Estabilidad Test (%)	Suavidad Test (%)	Robustez Test (%)
#25	91.56%	82.24%	70.42	96.29	72.62	97.44	92.55
#8	93.77%	79.75%	63.60	99.14	66.91	99.27	93.94
#9	96.91%	79.74%	71.38	95.31	65.10	96.60	82.54
#15	92.92%	74.82%	70.21	81.62	62.66	99.23	77.72
#21	85.02%	73.14%	52.10	98.92	59.38	90.11	93.06

Tabla 4.3: Configuración de hiperparámetros para los 5 modelos más destacados en la tarea de vuelo estacionario (*Hovering*).

Parámetro	Config. 8	Config. 9	Config. 15	Config. 21	Config. 25
Capas Ocultas Actor	[64, 64]	[128, 64]	[128, 128]	[256, 256]	[64, 64]
Capas Ocultas Crítico	[64, 64]	[64, 64]	[128, 128]	[64, 64]	[128, 128]
Activación	Tanh	ReLU	ReLU	Tanh	ReLU
Tasa de Aprendizaje	$1 \cdot 10^{-4}$	$1 \cdot 10^{-4}$	$3 \cdot 10^{-4}$	$1 \cdot 10^{-3}$	$3 \cdot 10^{-3}$
Clip Param PPO	0.2	0.1	0.1	0.1	0.1
Coefficiente Entropía	0.0	0.001	0.0	0.05	0.0
Ruido Inicial (σ)	0.2	1.0	0.2	0.1	0.8

El análisis constata que, si bien todos mantienen niveles altísimos de seguridad, es decir, mantienen el vuelo adecuadamente, los modelos divergen en su capacidad para ofrecer un vuelo suave frente a la necesidad de mantener un posicionamiento estricto. Así, la configuración 25 muestra el área más extensa y equilibrada. Su forma geométrica se aproxima a un pentágono regular y, lo más importante, esta figura se mantiene prácticamente inmutable al escalar el estrés del simulador desde 256 hasta 8196 entornos paralelos, demostrando una invarianza impecable ante el caos dinámico. Las configuraciones 8 y 9 se presentan como excelentes alternativas secundarias, sirviendo como demostración de que la estabilización

pura es un problema resuelto y parametrizable bajo estas métricas. Por consiguiente, la optimización para el refinamiento de trayectorias y maniobras complejas se delega a la siguiente fase de experimentación (*Tracking*).

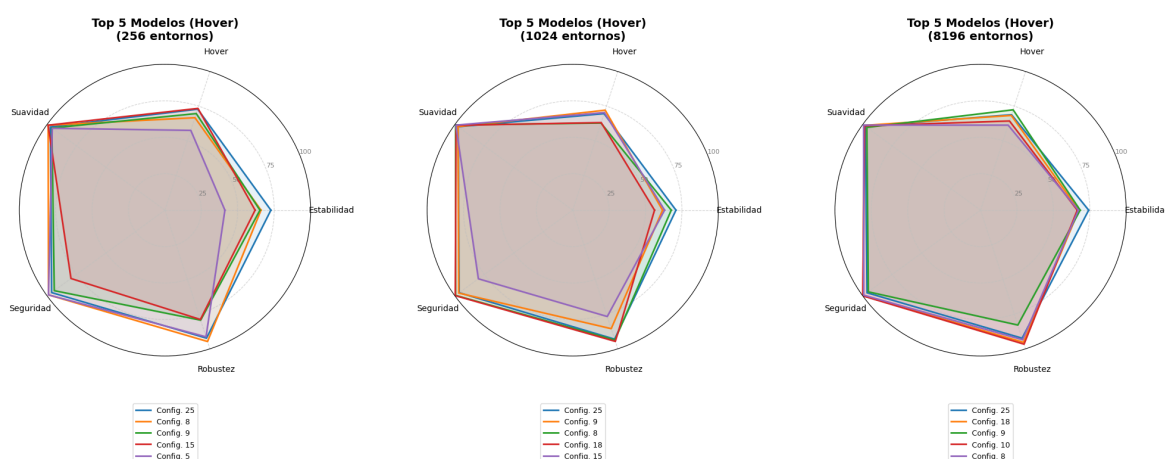


Figura 4.7: Análisis mediante gráficos de radar del rendimiento multidimensional. Se expone la invariabilidad de los mejores modelos frente al incremento masivo de instanciación paralela (256, 1024 y 8196 entornos de prueba).

4.3. Experimentos definidos para seguimiento del plan de vuelo

Tras haber validado la viabilidad de los experimentos obteniendo unos resultados satisfactorios en las pruebas de estabilidad, el entorno fue adaptado para llevar a cabo la experimentación del objetivo principal de este proyecto, que es el seguimiento dinámico de trayectorias. Para esta fase, se conservó íntegramente la arquitectura de automatización descrita previamente, reutilizando el flujo de búsqueda aleatoria para evaluar las 35 configuraciones generadas.

Así, se adaptó también el cálculo de la puntuación global, cuya composición final se refleja en la Tabla 4.4.

Tabla 4.4: Cálculo de las métricas en Tracking.

Dimensión Normalización	Fórmula Aplicada	Símbolo	Término evaluado	Límite (clip)	Rango Real
Seguimiento (35%) Rango: [0.0, 11.0]	$\mathcal{L}(\text{clip}(r_{pos}))$ + $\text{clip}(r_{track_vel})$ + $\text{clip}(r_{heading})$	r_{pos}	Precisión de posición	≤ 1.5	[0, 2.0]
		r_{track_vel}	Velocidad de persecución	≤ 7.0	[0, 8.0]
		$r_{heading}$	Alineación de rumbo	≤ 2.5	[0, 3.0]
Seguridad (25%) Rango: [0.0, 1.0]	$1 - (t_{alt} + t_{att} + t_{safe})$	t_{alt}	Pérdida de altitud	—	[0, 1]
		t_{att}	Inclinación crítica	—	[0, 1]
		t_{safe}	Protección del simulador	—	[0, 1]
Estabilidad (20%) Rango: [0.0, 3.0]	$r_{att_stab} + r_{ang_stab}$	r_{att_stab}	Estabilidad de actitud	—	[0, 2.0]
		r_{ang_stab}	Estabilidad velocidad angular	—	[0, 1.0]
Robustez (10%) Rango: [0.0, 1.0]	$CV = \frac{\sigma}{\mu}$	CV	Coefficiente de variación	—	[0, 1.0]
Suavidad (10%) Rango: [0.0, 0.016]	$-r_{act}$	r_{act}	Acción de control	—	[-0.016, 0]

4.4. Resultados de la experimentación para el seguimiento del plan de vuelo

El análisis inicial de convergencia sobre la recompensa media (Figura 4.8) revela que la mayoría de los modelos convergentes se ubican entre los 250 y 300 puntos.

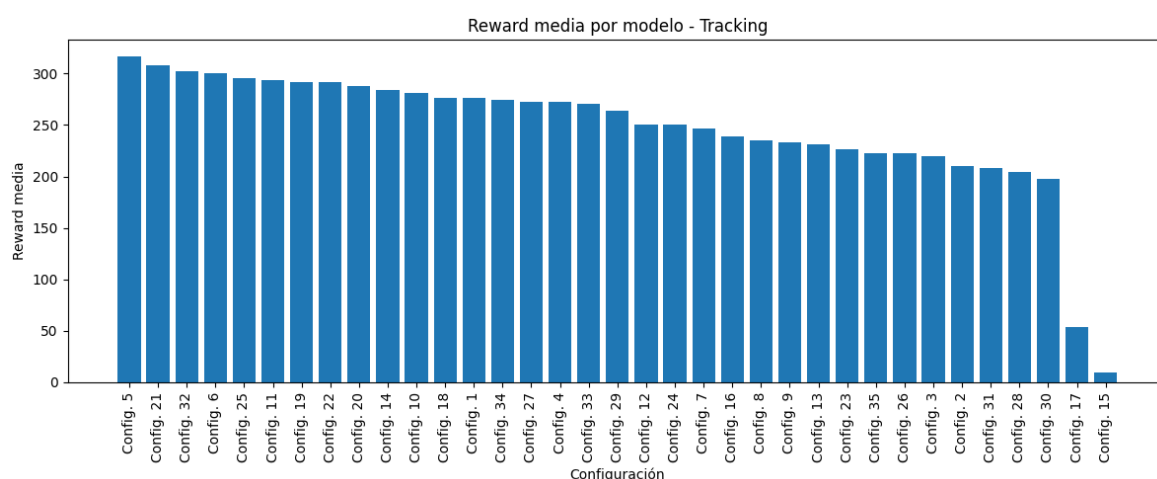


Figura 4.8: Distribución de la recompensa media obtenida por cada uno de los 35 modelos durante la fase de entrenamiento de *tracking*.

Al aislar los 5 mejores modelos, su evolución temporal (Figura 4.9) muestra un comportamiento estable. A diferencia de la fase de estabilización, las 5 políticas superan los 250 puntos de recompensa antes de las 200 iteraciones, evidenciando que los hiperparámetros seleccionados aceleraron la asimilación del entorno.

Como se aprecia en la Figura 4.10, el sobreajuste presente en la fase de evaluación provoca una ligera degradación de la recompensa. Para determinar el verdadero impacto de esta caída de recompensa se ha calculado la puntuación global adaptada a la métrica prioritaria de seguimiento, visible en la Tabla 4.5. A pesar de sufrir una caída cercana al 28% en la recompensa entre entrenamiento y validación, la configuración 32 es el mejor modelo encontrado con una puntuación global de 79.67%. Esta política logra el mejor seguimiento del objetivo (75.53%) y una suavidad de maniobra impecable (99.39%), resolviendo con éxito el compromiso entre agresividad aerodinámica y control físico. Sin embargo, también podría tomarse el modelo 14 como modelo óptimo debido a que posee una tasa de éxito superior (92.36%), si bien presenta una ligera reducción de la suavidad durante el vuelo (84.63%).

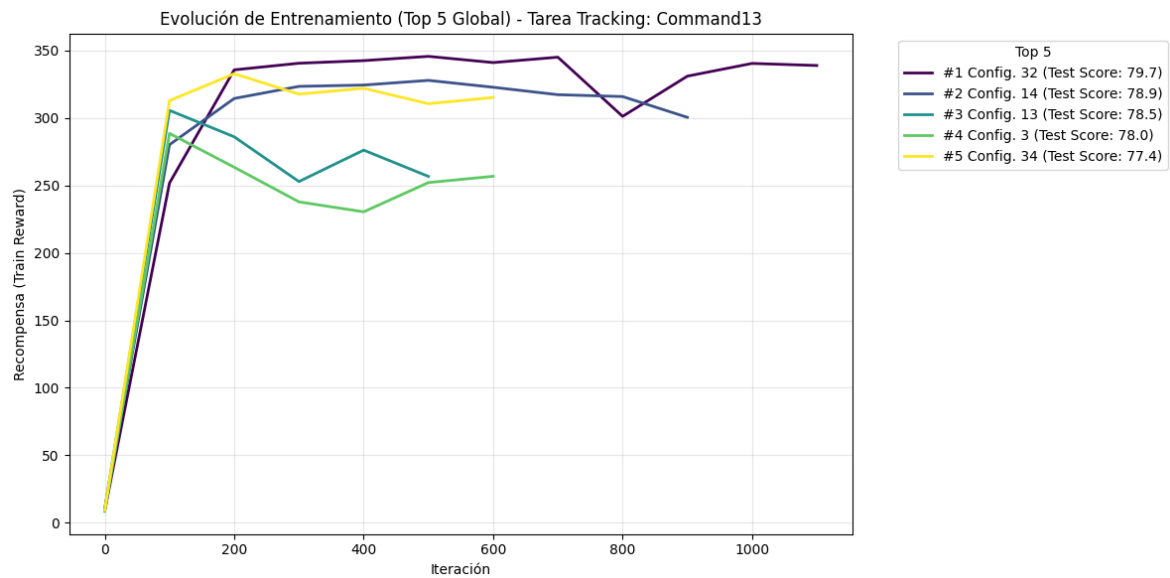


Figura 4.9: Curvas de evolución temporal de la recompensa durante el entrenamiento, filtradas para los cinco mejores modelos resultantes de la evaluación de *Tracking*.

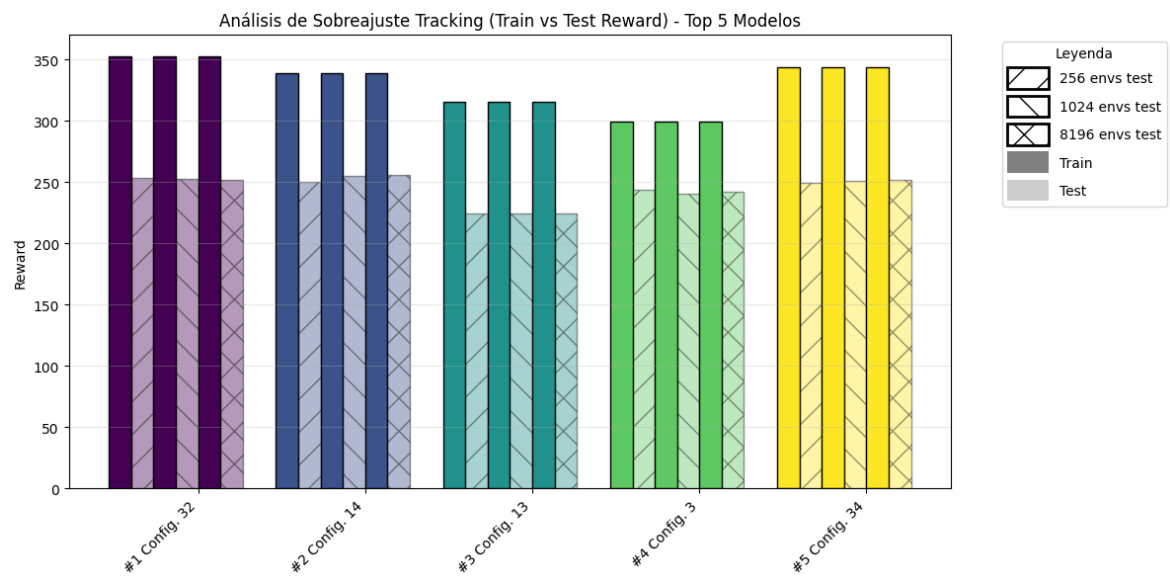


Figura 4.10: Análisis del sobreajuste en seguimiento dinámico: comparativa entre la recompensa de entrenamiento y el rendimiento en validación.

Tabla 4.5: Resultados globales de la evaluación multidimensional para los 5 mejores modelos de seguimiento (*Tracking*), ordenados por su *Score Global* en Test.

Modelo	Score Train (%)	Score Test (%)	Tracking Test (%)	Seguridad Test (%)	Estabilidad Test (%)	Suavidad Test (%)	Robustez Test (%)
#32	94.52%	79.67%	75.53	88.44	68.06	99.39	75.73
#14	85.06%	78.86%	72.50	92.36	68.40	84.63	82.51
#13	93.08%	78.53%	73.09	87.81	67.52	99.76	75.17
#3	81.52%	78.04%	73.38	94.84	71.88	60.31	82.41
#34	83.68%	77.43%	73.12	86.70	67.11	91.26	76.16

La Tabla 4.6 detalla la configuración de estos 5 modelos. Destaca el uso de arquitecturas muy profundas (redes con capas de 256 y 128 neuronas) y funciones de activación como *GELU* y *Tanh*. El mejor modelo (el 32) combinó una tasa de aprendizaje de $3 \cdot 10^{-4}$ y una entropía mínima (0.001) con un alto ruido de exploración ($\sigma = 0.8$), lo que le permitió descubrir maniobras robustas subóptimas desde etapas tempranas.

Tabla 4.6: Configuración de hiperparámetros para los cinco modelos más destacados en la tarea de seguimiento de trayectorias dinámicas (*Tracking*).

Parámetro	Config. 32	Config. 14	Config. 13	Config. 3	Config. 34
Capas Ocultas Actor	[256, 256]	[128, 64]	[64, 64]	[128, 128]	[128, 128]
Capas Ocultas Crítico	[256, 256]	[64, 64]	[256, 256]	[64, 64]	[64, 64]
Activación	GELU	GELU	Tanh	Tanh	GELU
Tasa de Aprendizaje	$3 \cdot 10^{-4}$	$5 \cdot 10^{-5}$	$1 \cdot 10^{-4}$	$1 \cdot 10^{-3}$	$5 \cdot 10^{-5}$
<i>Clip Param</i> PPO	0.1	0.2	0.3	0.2	0.2
Coeficiente Entropía	0.001	0.05	0.0	0.05	0.01
Ruido Inicial (σ)	0.8	0.2	0.8	0.8	0.5

El perfil multidimensional (Figura 4.11) revela que la arquitectura de aprendizaje por

refuerzo asimila con éxito la dinámica de vuelo, destacando el modelo 32 por su capacidad para igualar velocidades y anticipar giros maximizando la suavidad del control.

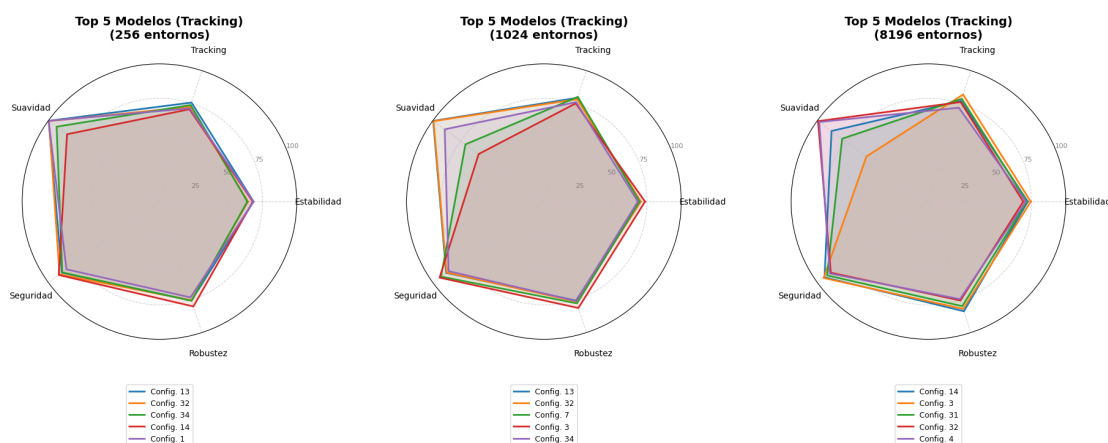


Figura 4.11: Análisis mediante gráficos de radar para evaluar la eficacia predictiva, el desempeño dinámico y la consistencia operativa de los modelos de *Tracking* al escalar el estrés computacional (256, 1024 y 8196 entornos).

Esta eficacia para seguir las rutas de forma autónoma se ve reflejado en la Figura 4.12, donde se representa el rendimiento de un dron promedio del modelo 32. Se aprecia que el dron logra corregir un desplazamiento inicial de 6 metros hasta estabilizar el error entre 3 y 4 metros, que es un nivel de precisión satisfactorio dado que el modelo de aerotaxi tiene 5 metros de envergadura y que el error se mide con respecto al centro de masas del mismo.

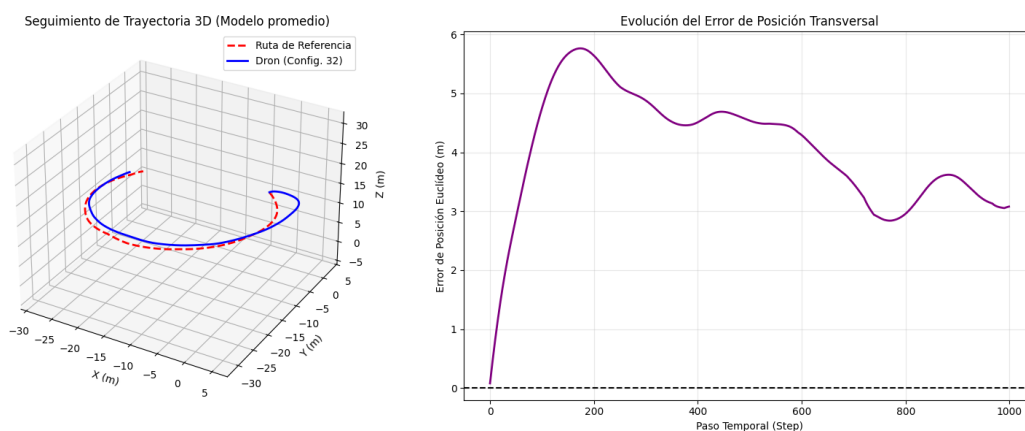


Figura 4.12: Seguimiento de una ruta aleatoria.

5. Conclusiones

El presente proyecto ha culminado con la resolución exitosa de los dos grandes desafíos cinemáticos planteados originalmente: la estabilización estática de la aeronave (*hovering*) y el seguimiento dinámico de trayectorias estocásticas (*tracking*). A pesar de la enorme dificultad inherente a la optimización en espacios continuos de alta dimensionalidad, los modelos han convergido hacia comportamientos precisos, demostrando la viabilidad del aprendizaje por refuerzo profundo para el control autónomo de aeronaves de gran envergadura.

Dada la extrema no linealidad del sistema, el sobreajuste observado resulta un fenómeno esperable. La literatura científica [Dulac-Arnold et al., 2021, Zhao et al., 2020] documenta ampliamente esta brecha de transferencia (*Sim-to-Real gap*), avalando que técnicas como la aleatorización de dominio sacrifican rendimiento teórico para garantizar robustez frente a perturbaciones. Teniendo esto en cuenta, el agente ha conservado su viabilidad operativa y capacidad de generalización en todos los ensayos, confirmando la solidez de las políticas asimiladas.

Desde el punto de vista de la investigación, este trabajo ha demostrado la viabilidad de implementar un controlador autónomo de drones aerotaxi de gran escala en persecución tridimensional empleando aprendizaje por refuerzo. Para ello se ha creado un *pipeline* de experimentación totalmente automatizado sobre Isaac Lab, cuyo desarrollo y validación ha requerido de aproximadamente 150 horas de trabajo, empleando una tarjeta gráfica NVIDIA RTX 5080 y 64 GB de memoria RAM.

Así, se han abordado todos los objetivos que se propusieron al inicio de este proyecto:

1. **Objetivo 1:** Se ha desarrollado un modelo de aprendizaje por refuerzo que recibe como entrada el estado de la aeronave y el plan de vuelo, que es la ruta generada en tiempo real, permitiendo su funcionalidad así con rutas realistas que se puedan adaptar

a cualquier condición inesperada en el instante de tiempo actual, generando a partir de estos valores las velocidades angulares de los rotores.

2. **Objetivo 2:** Se ha implementado un *pipeline* dentro del simulador Isaac Lab que permite entrenar el controlador de vuelo en condiciones realistas con aleatorización de viento, masa, posición, rotación del dron y percepción de ruido en los sensores del dron.
3. **Objetivos 3 y 4:** Se ha llevado a cabo un análisis del éxito de la tarea (seguridad), la capacidad predictiva (seguimiento), la estabilidad, la suavidad y la robustez frente a la aleatorización de dominio y la modificación de la ruta en tiempo real por medio de la definición y aplicación de una métrica de puntuación final que combina de forma ponderada estas 5 dimensiones, alcanzando el mejor modelo una puntuación total de 79.67 sobre 100 en la fase de evaluación.

Por otro lado, se han desarrollado las distintas competencias enumeradas al comienzo de esta memoria, ya que se han aplicado conceptos, técnicas y métodos de Inteligencia Artificial en un entorno de aprendizaje por refuerzo profundo, planificando y ejecutando este proyecto adecuadamente en un entorno que requiere de un controlador inteligente que interactúa con el entorno físico simulado, y los resultados han sido analizados con técnicas de estadística para poder determinar cuál o cuáles son los mejores modelos obtenidos, mostrando una visualización que permita una comprensión profunda e intuitiva del entrenamiento y la evaluación realizadas.

Como líneas de trabajo futuro, se propone extender la experimentación para minimizar el sobreajuste y facilitar la validación en prototipos físicos reales, evaluando el consumo energético y las latencias de hardware. Paralelamente, se podría integrar un módulo que permita que los drones esquiven obstáculos estáticos y dinámicos, incluyendo detección por LiDAR que modifique el plan de vuelo en tiempo real. Las políticas generadas por el entrenamiento realizado podrían ajustarse a estas rutas adecuadamente, debido a que los drones se han entrenado con rutas que se generan modificando la posición, velocidad y rotación en cada instante de tiempo.

Por último, declaro haber utilizado herramientas de inteligencia artificial para ajustar la redacción y el formato de las tablas de esta memoria. Dicho empleo se ha llevado a cabo supervisando y corroborando individualmente cada sugerencia de cambio. De este modo, aseguro el mantenimiento de mi autoría y el respeto a las ideas originales del trabajo.

A. Anexos

A.1. Código del proyecto

Todo el código al que se hace referencia en este trabajo se encuentra en la siguiente dirección de Github: <https://github.com/I3A-NavSys/navsim/tree/Teresabbranch>.

A.2. Otros experimentos realizados

Se realizaron múltiples experimentos durante la fase de entrenamiento para el problema de estabilidad y el de seguimiento de rutas. A continuación, y a modo representativo, se incluyen algunos de los experimentos que se realizaron para el problema de estabilidad en las últimas fases de experimentación del mismo, hasta la obtención de una política aceptable que todavía no sería la final, como se detalla en las Figuras A.1 a A.4.

Por otro lado, en el problema de seguimiento de rutas se realizó una experimentación similar, donde los mayores hallazgos fueron la necesidad de aumentar la fuerza total de empuje, una experimentación extensa con múltiples términos de recompensa, partiendo de los obtenidos al final en el problema de estabilidad, y la necesidad de incorporar visión de futuro en las observaciones de la aeronave. Los resultados obtenidos son similares respecto a los que se obtuvieron en la experimentación de hover, comenzando con episodios de terminación temprana y baja recompensa. Se han seleccionado una serie de experimentos como muestra representativa, y sus resultados se pueden observar en la Figura A.5.

Experimento	Entorno	Reward total	Steps (duración episodio)	% se caen	% rotación excesiva	Escala de acción	Trust	Funciones de reward	Pesos	Sid	Observaciones del Actor	History length	Observaciones del Tarea	Exito	
Env_11:1	Masa 2000 +10%, centro de masas en 0.0 y clamp de acciones -2.2 actor y crítico basados [64,64], 128 steps, 60º de giro máximo, con early stopping	36.27	9371	100	0		30No aplica	Alive Action rate Terminating rev_pos_diff (exponencial) rev_ang_vel_z_diff_fine_gral net (yaw)	15 -0.01 -300 15 2		distance lin_vel ang_vel roll pitch yaw			No	
Env_11:2	Acciones 1,1, cerradas en 0 hover -0 bajar -0 subir	2	36.65	0	100	Config Env_11:1	No aplica	Alive Action rate Terminating rev_pos_diff x y rev_pos_diff z rev_vel z rev_ang_vel z rev_ang_vel_z_diff_fine_gral net (yaw)	-2 -0.01 -300 6 -4 2 2 0.25			Config Env_11:1	Config Env_11:1	No	
Env_11:3	Config Env_11:2	35.95	93.42	96.47	3.53	Config Env_11:1	No aplica	Config Env_11:1 Action rate Terminating rev_pos_diff z rev_vel z rev_pos_diff (exponencial) rev_ang_vel_z_diff_fine_gral net (yaw)	Env_11:1 -0.01 -1000 3 -4 5 2 0.25		distance lin_vel ang_vel height	Config Env_11:1	Config Env_11:1	Hover	No
Env_11:4	Config Env_11:2	-16.62	37.15	100	0	Config Env_11:1	No aplica	bit_penalty_pg Alive Action rate Terminating rev_pos_diff z rev_vel z rev_pos_diff (exponencial) rev_ang_vel_z_diff_fine_gral net (yaw)	-5 15 -0.01 -1000 3 -4 5 2 0.25		projected_gravity		projected_gravity	Hover	No
Env_11:5	Config Env_11:1	-7.93	43.93	0	100	Config Env_11:1	No aplica	bit_penalty_pg Alive Terminating rev_pos_diff z rev_vel z rev_pos_diff (exponencial) rev_ang_vel_z_diff_fine_gral net (yaw)	-5 15 -50 3 -4 5 2 0.25		Config Env_11:4	Config Env_11:4	Config Env_11:4	Hover	No
Env_11:6	Config Env_11:1	10.9	42.62	0	100	Config Env_11:1	No aplica	bit_penalty_pg Alive Terminating rev_pos_diff z rev_vel z rev_pos_diff (exponencial) rev_ang_vel_z_diff_fine_gral net (yaw)	-5 15 -50 3 -4 5 2 0.25		Config Env_11:4	Config Env_11:4	Config Env_11:4	Hover	No
Env_11:7	Config Env_11:1	28.39	95.4	100	0	Config Env_11:1	No aplica	bit_penalty_pg Alive Action rate Terminating rev_pos_diff (exponencial) rev_ang_vel_z_diff_fine_gral net (yaw)	-5 15 -0.01 -300 15 2 0.25		Config Env_11:4	Config Env_11:4	Config Env_11:4	Hover	No

Figura A.1: Parte I. Selección de experimentación realizada para hover.

[illegible]

Env_5+3	Masa 2000 +/-10%, centro de masas en 0.00 y campo de acciones -2.2, actor tamaño 1128.128 y crítico tamaño 1256.256). 32 steps, terminación por rotación excesiva menos exigente (100° en lugar de 60°), número fijo de entornos a 2048, sin early stopping	-5.59	20.74	0	100	Config Env_11-1	No aplica	Action rate rev_pos_fine rev_hover_stability rev_pos_diff_cued rev_vel rev_ang_vel tilt_penalty_pg	-0.01 1 2 -2 -0.8 -2 -1.5	Config Env_5- Env_5+3	Config Env_11-4	Config Env_11-4	Config Env_11-4	Hover	No
Env_5+4	Config Env_5+3	-44.05	60.12	100	0	10	No aplica	Config Env_5+3	N3	Config Env_5- Env_5+3	Config Env_11-4	Config Env_11-4	Config Env_11-4	Hover	No
Env_6+1	Masa 2000 +/-10%, centro de masas en 0.00 y campo de acciones -2.2, actor tamaño 1128.128 y crítico tamaño 1256.256). 32 steps, terminación por rotación excesiva menos exigente (100° en lugar de 60°), número fijo de entornos a 2048, sin early stopping	-5.73	30.2	0	100	Config Env_11-1	No aplica	Config Env_5+3	N3	Config Env_5- Env_5+3	Config Env_11-4	Config Env_11-4	Config Env_11-4	Hover	No
Env_6+2	Config Env_6+1	-5.48	46.67	0	100	Config Env_11-1	No aplica	Action rate rev_pos_fine rev_hover_stability rev_pos_diff_cued rev_vel rev_ang_vel tilt_penalty_pg	-0.01 0.5 3 -1 -0.4 -0.8 -3	Config Env_11-4	Config Env_11-4	Config Env_11-4	Config Env_11-4	Hover	No
Env_6+3	Config Env_6+1	-7.12	48.21	0	100	Config Env_11-1	No aplica	Action rate rev_pos_fine rev_hover_stability rev_pos_diff_cued rev_vel rev_ang_vel tilt_penalty_pg	-0.01 0.3 3 -1 -0.4 -1.2 -4	Config Env_11-4	Config Env_11-4	Config Env_11-4	Config Env_11-4	Hover	No
Env_6+4	Config Env_6+1	-28.98	89.08	100	0	Config Env_11-1	No aplica	Action rate rev_pos_fine rev_hover_stability rev_pos_diff_cued rev_vel rev_ang_vel tilt_penalty_pg rev_attitude_stab	-0.01 0.5 3 -1 -0.4 -1.2 -4 3	Config Env_11-4	Config Env_11-4	Config Env_11-4	Config Env_11-4	Hover	No
Env_6+5	Config Env_6+1	-7.44	48.51	1.28	98.78	Config Env_11-1	No aplica	Action rate rev_pos_fine rev_hover_stability rev_pos_diff_cued rev_vel rev_ang_vel tilt_penalty_pg rev_attitude_stab rev_attitude_hnd	-0.01 0.5 3 -1 -0.4 -1.2 -4 3 4	Config Env_11-4	Config Env_11-4	Config Env_11-4	Config Env_11-4	Hover	No
Env_6+6	Config Env_6+1	-26	90	100	0	Config Env_11-1	No aplica	Action rate rev_pos_fine rev_hover_stability rev_pos_diff_cued rev_vel rev_ang_vel tilt_penalty_pg rev_attitude_stab rev_attitude_hnd	-0.01 0.5 3 -1 -0.4 -1.2 -3.5 4 3	Config Env_11-4	Config Env_11-4	Config Env_11-4	Config Env_11-4	Hover	No

Figura A.3: Parte III. Selección de experimentación realizada para hover.

Figura A.4: Parte IV. Selección de experimentación realizada para hover.

Experimento	Entorno	Fuerza de Empuje	Rwd Train	Rwd Test	Dif (%)	Funciones Reward	Pesos	Sid Train	Sid Test	Obs. Actor	History Length	Caída (%)	Exceso Rotación (%)	Entro	
	128 steps. Acciones clamp [-1,1] Masa +-10%. Viento aleatorio. Valla virtual 45m.	Base Actb (Hover) + Thrust scale 0.5. Max 12000 N.	261.50	103.15	-60.55%	track_pos, track_vel, attitude_stab3, ang_vel_stab5, action_rate	12.0, 4.0, 2.0, 1.0, 0.8	69.08	67.18(ruido)	dis, lin_vel, ang_vel, prol_grav, target_vel (con ruido)	318, 20%	12.10%	No (~69.6%)		
command9_11	128 steps. Acciones clamp [-1,1] Masa +-10%. Viento aleatorio. Valla scale 0.5. Max 12000 N.	Base Actb (Hover) + Thrust scale 0.5. Max 12000 N.	301.68	228.08	-24.39%	attitude_stab3, ang_vel_stab5, track_vel, track_pos, action_rate, heading	2.0, 1.0, 10.0, 3.0, 0.8, 4.0	0.00	84.59 (Config. Command9_11)	Config. Command9_11	Config. Command9_11	13.54%	5.08%	Parcial (~81.4%)	
command10_11	128 steps. Acciones clamp [-1,1] Masa +-10%. Viento aleatorio. Valla 45m. Yaw rate max 0.06.	Base Actb (Hover) + Thrust scale 0.5. Max 12000 N.	348.84	257.45	-26.20%	attitude_stab3, ang_vel_stab5, track_pos, action_rate, heading, track_vel3	2.0, 1.0, 2.0, 0.1, 3.0, 8.0	19.68	50.16 (Config. Command9_11)	Config. Command9_11	Config. Command9_11	4.56%	2.61%	Si (~92.8%)	
command11_11	128 steps. Acciones clamp [-1,1] Masa +-10%. Viento aleatorio. Valla 45m. Yaw rate max 0.25.	Base Actb (Hover) + Thrust scale 0.5. Max 12000 N.	358.37	245.25	-31.57%			21.74	60.66(ruido)	lin_vel, ang_vel, prol_grav, target_vel, target_future, yaw_error (con ruido)	Config. Command11_11	03.63%	11.99%	Parcial (~84.4%)	
command12_29	128 steps. Acciones clamp [-1,1] Masa +-10%. Viento aleatorio. Valla 45m. Yaw rate max 0.25. Dropout y entropia y epsilon con decay	Base Actb (Hover) + Thrust scale 0.5. Max 12000 N.	353.79	236.53	-33.15%	Config. Command11_11	2.0, 1.0, 2.0, 0.1, 3.0, 8.0	21.33	68.75(ruido)	Config. Command12_2	Config. Command12_2	9.04%	15.53%	Parcial (~75.4%)	

Figura A.5: Selección de experimentación realizada para tracking.

A.3. Comportamientos detectados en la fase de entrenamiento

Durante la fase de entrenamiento tanto del problema de estabilización como del de seguimiento de rutas se identificaron comportamientos diversos que realizaban los drones al visualizar varias de las configuraciones con el simulador de Isaac Lab. Estos comportamientos, sin una configuración óptima de recompensas, observaciones, del entorno y de PPO, se trasladaba a las pruebas de evaluación. Así, entre las más frecuentes observadas encontramos las que se detallan en la Figura A.6.

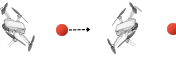
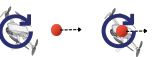
Comportamiento - Hover training	Representación Visual
Hover paralelo en un lado/encima/debajo	
Vuelo por el espacio	
Hover girando como una peonza (yaw)	
Caída lenta/No entrenar	
Entrenamiento óptimo	
Comportamiento - Tracking training	Representación Visual
Tracking paralelo por detrás de la bola mirando hacia atrás/hacia delante	
Tracking paralelo por delante de la bola mirando hacia atrás/hacia delante	
Tracking paralelo por la izquierda de la bola, mirando hacia izquierda o derecha	
Tracking paralelo por la derecha de la bola, mirando hacia izquierda o derecha	
Tracking sobre la bola en cualquier orientación	
Caída lenta/Vuelo por el espacio	
Tracking girando como una peonza con todas las combinaciones (yaw)	
Cualquiera de las anteriores con cualquier ángulo/No entrenar	
Entrenamiento óptimo	

Figura A.6: Comportamientos identificados durante la fase de entrenamiento en hover y tracking.

Referencia bibliográfica

- [Beard and McLain, 2012] Beard, R. W. and McLain, T. W. (2012). *Small unmanned aircraft: Theory and practice*. Princeton university press.
- [Bergstra and Bengio, 2012] Bergstra, J. and Bengio, Y. (2012). Random search for hyperparameter optimization. *Journal of machine learning research*, 13(2).
- [Cai et al., 2011] Cai, G., Chen, B. M., and Lee, T. H. (2011). *Unmanned rotorcraft systems*. Springer Science & Business Media.
- [Camacho and Bordons, 2013] Camacho, E. F. and Bordons, C. (2013). *Model Predictive Control*. Springer.
- [Casado et al., 2026] Casado, R., Bermúdez, A., Hernández-Orallo, E., and Calafate, C. T. (2026). A multi-constrained trajectory representation framework for predictable and low error uav missions. *Journal of Intelligent & Robotic Systems*.
- [Dulac-Arnold et al., 2021] Dulac-Arnold, G., Levine, N., Mankowitz, D. J., Li, J., Paduraru, C., Gowal, S., and Hester, T. (2021). Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning*, 110(9):2419–2468.
- [Farid et al., 2018] Farid, G., Hamid, H. T., Karim, S., and Tahir, S. (2018). Waypoint-based generation of guided and optimal trajectories for autonomous tracking using a quadrotor uav. *Stud. Inform. Control*, 27:225–236.
- [Henderson et al., 2018] Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., and Meger, D. (2018). Deep reinforcement learning that matters. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32.

-
- [Huang et al., 2025] Huang, J., Cui, Y., Xi, G., Bai, S., Li, B., Wang, G., and Neretin, E. (2025). Gtrxl-sac-based path planning and obstacle-aware control decision-making for uav autonomous control. *Drones*, 9(4):275.
- [Hwangbo et al., 2019] Hwangbo, J., Lee, J., Dosovitskiy, A., Bellicoso, D., Tsounis, V., Koltun, V., and Hutter, M. (2019). Learning agile and dynamic motor skills for legged robots. *Science robotics*, 4(26):eaau5872.
- [Hwangbo et al., 2017] Hwangbo, J., Sa, I., Siegwart, R., and Hutter, M. (2017). Control of a quadrotor with reinforcement learning. *IEEE Robotics and Automation Letters*, 2(4):2096–2103.
- [Jacinto et al., 2024] Jacinto, M., Pinto, J., Patrikar, J., Keller, J., Cunha, R., Scherer, S., and Pascoal, A. (2024). Pegasus simulator: An isaac sim framework for multiple aerial vehicles simulation. In *2024 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 917–922. IEEE.
- [Kaufmann et al., 2020] Kaufmann, E., Loquercio, A., Ranftl, R., Müller, M., Koltun, V., and Scaramuzza, D. (2020). Deep drone acrobatics. *arXiv preprint arXiv:2006.05768*.
- [Khalil and Grizzle, 2002] Khalil, H. K. and Grizzle, J. W. (2002). *Nonlinear systems*, volume 3. Prentice hall Upper Saddle River, NJ.
- [Kober et al., 2013] Kober, J., Bagnell, J. A., and Peters, J. (2013). Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274.
- [Koch et al., 2019] Koch, W., Mancuso, R., West, R., and Bestavros, A. (2019). Reinforcement learning for uav attitude control. *ACM Transactions on Cyber-Physical Systems*, 3(2):1–21.
- [Konda and Tsitsiklis, 1999] Konda, V. and Tsitsiklis, J. (1999). Actor-critic algorithms. *Advances in neural information processing systems*, 12.
- [Lee et al., 2020] Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., and Hutter, M. (2020). Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5(47):eabc5986.
- [Luukkonen, 2011] Luukkonen, T. (2011). Modelling and control of quadcopter. aalto university.
-

- [Mahony et al., 2012] Mahony, R., Kumar, V., and Corke, P. (2012). Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor. *IEEE robotics & automation magazine*, 19(3):20–32.
- [Nonami et al., 2010] Nonami, K., Kendoul, F., Suzuki, S., Wang, W., and Nakazawa, D. (2010). *Autonomous flying robots: unmanned aerial vehicles and micro aerial vehicles*. Springer Science & Business Media.
- [NVIDIA, 2026a] NVIDIA (2026a). Isaac lab documentation. <https://isaac-sim.github.io/IsaacLab>.
- [NVIDIA, 2026b] NVIDIA (2026b). Isaac sim documentation. <https://docs.isaacsim.omniverse.nvidia.com>.
- [NVIDIA, 2026c] NVIDIA (2026c). Nvidia isaac sim. <https://developer.nvidia.com/isaac-sim>.
- [NVIDIA, 2026] NVIDIA (2026). Nvidia isaac sim documentation. <https://developer.nvidia.com/isaac-sim>.
- [Park et al., 2021] Park, Y., Kim, W., and Moon, H. (2021). Time-continuous real-time trajectory generation for safe autonomous flight of a quadrotor in unknown environment. *Applied Sciences*, 11(7):3238.
- [Ray et al., 2019] Ray, A., Achiam, J., and Amodei, D. (2019). Benchmarking safe exploration in deep reinforcement learning. *arXiv preprint arXiv:1910.01708*, 7(1):2.
- [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- [Shi et al., 2024] Shi, D., Shen, J., Gao, M., and Yang, X. (2024). A multi-waypoint motion planning framework for quadrotor drones in cluttered environments. *Drones*, 8(8):414.
- [Sutton et al., 1998] Sutton, R. S., Barto, A. G., et al. (1998). *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge.
- [Tobin et al., 2017] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., and Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 23–30. IEEE.

[Vincent et al., 2018] Vincent, F.-L., Peter, H., Riashat, I., Marc, G. B., and Joelle, P. (2018). An introduction to deep reinforcement learning. *Foundations and Trends® in Machine Learning*, 11(3-4):219–354.

[Zhao et al., 2020] Zhao, W., Queralta, J. P., and Westerlund, T. (2020). Sim-to-real transfer in deep reinforcement learning for robotics: a survey. In *2020 IEEE symposium series on computational intelligence (SSCI)*, pages 737–744. IEEE.